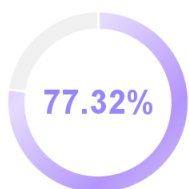


NO. b5366970920m9609 | 2026-04-08 22:29:41

- 题目：基于端到端小目标武器检测算法研究
- 作者：万楚峰
- 检测所属单位： -

📄 论文字符数：60656 📄 论文页数：60 📊 表格数量：4 🖼️ 图片数量：22

检测结果



77.32%

疑似AIGC生成占比

22.68%

人工撰写占比

结果分布

序号	章节	疑似AIGC生成文字/章节总字数	疑似AIGC生成章节占比	人工撰写占比
1	摘要	394/396	99.0%	1.0%
2	Abstract	180/227	79.0%	21.0%
3	第1章 引言	2883/2921	98.0%	2.0%
4	第2章 武器图像数据集	1198/1943	61.0%	39.0%
5	第3章 改进的YOLOv11模型设计	4817/6069	79.0%	21.0%
6	第4章 实验设计与结果分析	2405/4758	50.0%	50.0%
7	第5章 武器智能识别平台	1408/1635	86.0%	14.0%

*注:格式规范的情况下可准确识别章节，若论文中无章节，可能会识别有误。

片段分布

■ 疑似AIGC生成 □ 人工撰写



文字标注

■ 疑似AIGC生成 ■ 人工撰写



基于端到端小目标武器检测算法研究

北京理工大学珠海学院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

本科生毕业设计（论文）

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

基于端到端小目标武器检测算法研究

Research on End-to-End Small Object Weapon Detection Algorithm

学院：数理与土木工程学院

专业：数据科学与大数据技术

学生姓名：万楚峰

学号：221204102604

指导教师：周游宇

2026年5月1日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名： 日期： 年 月 日

关于使用授权的声明

本人完全了解北京理工大学珠海学院有关保管、使用毕业设计（论文）的规定，其中包括：A学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；A学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；A学校可允许本毕业设计（论文）被查阅或借阅；A学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；A学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名： 日期： 年 月 日 指导老师签名： 日期： 年 月 日

基于端到端小目标武器检测算法研究

摘 要

针对公共安全领域中小目标武器检测难、误报率高等问题，本文开展了基于端到端的小目标武器检测算法研究。结合交通枢纽、校园等高风险区域对实时高精度识别的实际需求，本文提出一种基于YOLOv11的融合模型，在骨干网络中引入空间与层次自注意力模块，提升全局特征建模能力；在检测头中集成精炼卷积模块，增强边界判别与定位精度，从而优化整体检测性能。

实验在Roboflow武器检测数据集上进行，结果显示该模型在mAP@0.5和mAP50-95上分别达到0.762和0.510，较YOLOv11n提升11.0和9.7个百分点，全面优于对比模型YOLOv11+BAR。消融实验与可视化分析进一步验证了各模块的有效性与模型的鲁棒性。为实现实用化部署，本文基于PyQt6搭建了一个集图像、视频与实时摄像头输入为一体的智能检测平台，集成识别、导出与可视化功能，为公共安全场景提供了高效、可视、可落地的技术解决方案。

关键词：武器检测；小目标识别；YOLOv11；自注意力机制；智能检测平台

Research on End-to-End Small Object Weapon Detection Algorithm

Abstract

To address the challenges of small-object weapon detection and high false alarm rates in the field of public security, this paper conducts a study on an end-to-end detection algorithm tailored for small weapon targets. Considering the urgent need for real-time and high-precision recognition in high-risk areas such as transportation hubs and campuses, we propose a fusion model based on YOLOv11. The model incorporates a Spatial and Hierarchical Self-Attention module in the backbone to enhance global feature representation, and integrates a Refinement Convolutional Module in the detection head to improve boundary discrimination and localization accuracy, thereby optimizing overall detection performance.

Experiments conducted on the Roboflow weapon detection dataset demonstrate that the proposed model achieves mAP@0.5 and mAP50-95 scores of 0.762 and 0.510, respectively, representing improvements of 11.0 and 9.7 percentage points over the baseline YOLOv11n model. The model also outperforms the comparative model YOLOv11+BAR across all key metrics. Ablation studies and visualization analyses further validate the effectiveness of each module and the robustness of the overall model. To facilitate practical deployment, an intelligent detection platform was developed using PyQt6, supporting image, video, and real-time camera input. The platform integrates detection, result export, and visualization functionalities, providing an efficient, intuitive, and deployable

Key Words: Weapon Detection Small Object Recognition YOLOv11 Self-Attention Mechanism
Intelligent Detection Platform

目 录

摘 要 I

Abstract II

1 引言 1

1.1 研究目的及意义 1

1.2 文献综述 1

1.3 研究思路 2

2 武器图像数据集 4

2.1 数据集介绍 4

2.2 数据预处理 5

2.2.1 归一化处理 5

2.2.2 数据增强 6

3 改进的YOLOv11模型设计 7

3.1 YOLOv11基础结构概述 7

3.2 SHSA模块设计与功能 8

3.3 RCM模块设计与功能 10

3.4 BAR模块设计与功能 12

3.5 融合模型 13

4 实验设计与结果分析 15

4.1 实验设计 15

4.2 实验结果 15

4.3 消融实验分析 18

4.4 对比实验分析 19

5 武器智能识别平台 21

结论 24

参考文献 25

附录 26

致谢 55

第1章 引言

1.1 研究目的及意义

随着社会治安形势的复杂化，武器类目标在公共场所出现的风险持续上升，对智能安防系统提出了更高的实时性与准确性要求。作为安防监控中的关键任务，武器检测的精度与响应速度直接关系到突发事件的预警能力和处置效率，尤其在复杂背景、遮挡和小目标场景中，传统方法难以满足实际需求。在此背景下，本文聚焦于小目标检测的技术难点，提出基于YOLOv11的优化模型，集成矩形自校准模块（RCM）与单头自注意力机制（SHSA），以提升模型的特征提取能力和鲁棒性，不仅具有重要的工程应用价值，也为目标检测任务的结构优化与轻量化设计提供了理论支持。

在具体研究中，本文以构建端到端高性能武器检测模型为目标，围绕算法设计、模块集成、性能评估与系统部署展开系统研究。通过引入RCM模块加强前景特征建模能力，引入SHSA模块提升多尺度特征感知能力，在提升精度的同时保持良好的计算效率。研究通过对比实验与消融实验验证模型有效性，并最终实现模型部署，提升其在智能安防场景中的实际应用能力。该研究不仅有助于提高小目标武器检测的精度与稳定性，也为后续相关应用提供了可参考的技术路径。

1.2 文献综述

近年来，随着深度学习和计算机视觉技术的迅猛发展，目标检测已成为学术界和工业界的重要研究方向，尤其广泛应用于安防、无人驾驶、智能监控等领域。其中，在安防场景中，公共区域如车站、机场、校园等地的人群密集性和突发风险性，对智能识别系统提出了更高的实时性与精度要求。目标检测作为安防系统中的核心技术，承担着对潜在威胁物体（如刀具、枪支等）进行快速准确识别的任务。

尽管传统目标检测方法在处理大尺寸、清晰目标方面已取得较好效果，但在实际安防场景中常面临诸多复杂因素干扰，如背景杂乱、光照变化、摄像头角度受限以及目标遮挡等问题，导致小目标检测准确率显著下降。此外，监控图像中的小目标本身尺寸有限，包含的判别信息稀缺，在多尺度融合与上下文理解方面对检测模型提出更高要求。因此，提升复杂环境下的小目标检测能力已成为当前研究的关键方向。

在深度学习推动下，基于卷积神经网络（CNN）的目标检测技术迅速发展。Deng等人[1]于2009年构建了大规模图像数据库ImageNet，奠定了数据驱动方法的基础；Goodfellow等人[2]在2016年提出的《Deep Learning》系统化总结了神经网络的发展，为视觉任务的模型设计提供了理论支撑。Redmon等人[3]于2016年提出YOLOv1模型，首次实现了端到端的实时目标检测。随后，Bochkovskiy等人[4]于2020年发布的YOLOv4引入CSPDarknet53主干网络与多种优化机制，在精度与速度之间取得平衡。Varghese等人[5]在2024年提出YOLOv8，引入C2f模块和改进的FPN结构，提升了多尺度特征融合能力。Wang与Zhang[6]于2024年发布的YOLOv11则在YOLOv8基础上集成C3k2、C2PSA和SPPF模块，进一步增强了对小目标的检测效果。

面向武器检测等安防关键场景，学者们也开展了针对性研究。Wang等人[7]于2022年基于YOLOv4构建了适用于CCTV系统的改进模型，提升了检测准确率和鲁棒性；Ashraf等人[8]于2022年结合YOLOv5s与CNN设计出适用于视频监控的实时系统；同年，Soban等人[9]通过参数和结构优化增强了在真实场景下的稳定性；Li与Chen[10]于2024年基于YOLOv8实现了一套高效的实时检测方案；Yadav等人[11]于2024年提出SIRD-YOLO模型，引入空间交互机制以提升复杂背景下的识别精度。

为进一步提升特征表达能力，研究者还尝试将注意力机制引入目标检测框架。Transformer架构由Vaswani等人[12]于2017年提出，为全局建模提供新思路。Woo等人[13]于2018年提出的CBAM模块结合通道与空间注意力，有效增

强了显著区域的响应；Zhu等人[14] 于2023年提出的BAR模块通过稀疏注意力与动态路由策略提升了全局建模能力；Yun与Ro[15]于2024年提出的SHSA结构通过单头注意力机制在轻量化前提下增强模型感知力；Wang等人[16]于2024年提出的RCM模块利用轴向上下文建模加强了前景特征提取，尤其适用于小目标检测。

尽管现有方法在性能与效率方面取得积极进展，但面对遮挡、复杂背景、小尺寸物体等实际场景仍存在精度瓶颈。为此，本文在YOLOv11基础上引入RCM与SHSA模块，通过增强多尺度感知与上下文理解能力，进一步优化对小型武器等隐蔽目标的检测性能。相较于Wang等人[7]和Yadav等人[11]的方法，本文模型在鲁棒性、适应性与工程部署效率方面展现出更高的综合优势。

1.3 研究思路

本研究聚焦于提升武器目标在复杂场景下的检测精度与系统实用性，围绕三个核心问题展开系统设计与实验验证：

首先，针对小目标在复杂背景中易被遮挡、轮廓模糊以及视觉特征不显著等问题，研究通过构建多样化的数据增强策略予以缓解。具体包括Mosaic拼接、几何变换、颜色扰动等方法，以扩充样本特征分布、提高模型对小目标的识别鲁棒性，增强对不同光照、角度和尺度条件下武器目标的适应能力。

其次，围绕YOLOv11的结构优化问题，设计并集成两种功能模块：单头空间与层级注意力机制（SHSA）与矩形自校准模块（RCM）。SHSA模块引入轻量级注意力机制以提升模型的全局建模能力，特别适用于小目标与复杂背景的特征提取；RCM模块则通过轴向上下文建模与形状校准机制，有效增强检测头部对目标边界的表征能力。通过消融实验与对比实验，验证该融合架构在提高目标检测精度与鲁棒性方面的显著优势。

最后，为推动算法在实际场景中的应用落地，研究构建了基于PyQt6的武器智能检测平台，实现模型部署的可视化交互。平台支持图像、视频与实时摄像头多源输入，具备一键识别、结果导出、日志记录等功能，界面友好、操作简便，具有良好的工程可扩展性与应用价值。

图1展示了本研究的技术路径与模块设计，涵盖从数据获取、模型改进、实验验证到平台实现的全过程。

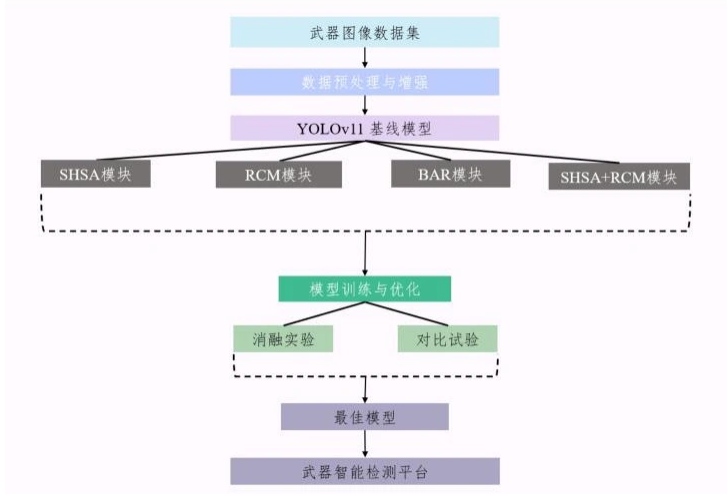


图1 研究思路框架图

论文后续章节安排如下：

第二章介绍武器图像数据集的基本情况。阐述所采用数据的来源与组成，说明图像标注格式转换、归一化处理与数据增强等关键预处理流程。

第三章详细解析YOLOv11的基础结构与模型改进方案。重点介绍SHSA与RCM模块的设计原理与集成方法，分析其在特征提取与边界定位中的性能优势。

第四章开展实验结果分析与对比。基于多项评价指标（包括Precision、Recall、F1-score与mAP）对各模型在测试集上的性能表现进行全面评估，并通过消融实验与对比实验验证所提模型的有效性。

第五章搭建武器智能识别平台。介绍平台的功能结构、用户界面与使用流程，展示模型在图像识别、视频检测与实时监控等多场景下的应用效果。

第六章总结研究成果并提出未来展望。归纳本研究在模型设计与平台实现方面的主要贡献，分析存在的局限性，提出未来在模型轻量化、跨场景适应性与平台多模态融合方面的优化方向。

第2章 武器图像数据集

2.1 数据集介绍

研究采用公开的Roboflow武器检测数据集，该数据集共包含2028张图像，涵盖三类常见武器目标：Gun（枪）、Rifle（步枪）和Knife（刀具）。图像分辨率较高，目标位置清晰明确，且每张图像均附带有标准的边界框标注信息，能够准确标示目标的位置、尺寸及类别，满足目标检测任务对高质量标注数据的基本要求。图2展示了三种类别的典型图像示例，从中可以看出不同类别在形状、颜色和尺寸上的多样性，这对模型判别提出了较高的感知要求。



(a)Gun (b)Rifle (c)Knife

图2 目标示例

其实例数量分布如图3所示。从数据统计可知，Gun和Rifle类的样本数量相对接近，均为约600张图像，而Knife类目标稍少，仅为400张左右，类别分布较为均衡，避免了模型训练过程中某一类别的过拟合问题。该数据集场景丰富，目标大小跨度大，背景复杂程度不一，涵盖了室内外不同环境下的拍摄样本，具备较强的代表性与挑战性。模型在该数据集上训练和评估结果，可较好反映其在实际多场景下部署的适应能力。

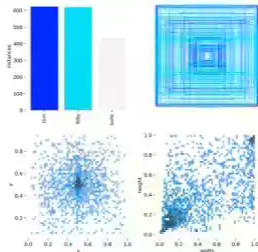


图3 数据集类别实例分布及边界框信息

2.2 数据预处理

为确保武器检测数据集能够适配YOLO系列模型的输入规范，需对原始数据进行系统化的预处理操作。主要步骤

包括：标注格式的统一转换、边界框坐标的归一化处理、训练所需标签文件的生成、数据集的合理划分以及多种数据增强策略的应用。上述预处理流程不仅有助于提升模型的训练效率，还显著增强了其在实际应用中的泛化能力，使其更适应复杂场景中的小目标识别与边界判别任务，特别适用于枪支、刀具、步枪等典型武器在模糊、遮挡与多目标干扰场景中的智能检测需求。

2.2.1 归一化处理

借助Roboflow平台提供的标注信息，对数据格式进行转换，使其符合YOLO模型的标签规范。YOLO所需的标签格式如下所示：

`<class_id> <x_center> <y_center> <width> <height>`

在完成原始标注信息解析后，需要对每个目标的边界框坐标进行归一化处理，使其符合YOLO模型的输入格式要求。YOLO模型采用相对于图像尺寸的归一化坐标系，因此必须将边界框的坐标值转换为图像宽度与高度的比例值。具体而言，边界框中心点的横纵坐标计算公式如下：

$$x_{center} = \frac{x_{min} + x_{max}}{2 \times image_width} \quad (1)$$

$$y_{center} = \frac{y_{min} + y_{max}}{2 \times image_height} \quad (2)$$

边界框的宽度与高度则通过如下公式计算：

$$width = \frac{x_{max} - x_{min}}{image_width} \quad (3)$$

$$height = \frac{y_{max} - y_{min}}{image_height} \quad (4)$$

上述归一化计算确保了所有坐标值均被标准化至[0, 1]区间，从而满足YOLO输入标签的规范。这种归一化机制不仅压缩了输入数据的动态范围，也提高了模型在不同分辨率图像间的迁移能力，使模型在推理时能兼容多种分辨率输入，提升适配性。

归一化后的标签信息会被自动写入每张图像对应的标签文件中，保证训练过程中的高效数据读取与批处理能力。此外，数据集依据70%：20%：10%的比例被划分为训练集、验证集与测试集三部分。该划分策略可确保模型训练过程中的数据独立性与评估的客观性，避免因样本交叉污染而造成精度虚高，提升整体建模可靠性。以上所有处理流程均通过Roboflow平台在线工具自动执行，极大减少了手动操作带来的格式误差与一致性问题，也为后续部署奠定了标准化数据结构基础。

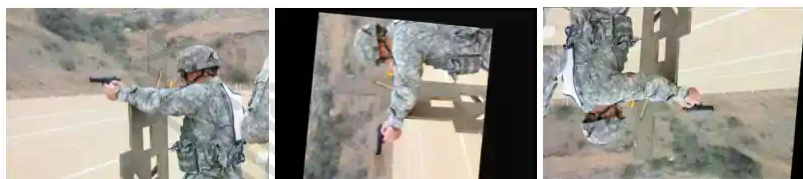
2.2.2 数据增强

在目标检测任务中，数据增强是提升模型鲁棒性与泛化能力的关键手段，尤其在样本数量有限或样本分布不均的情况下更显重要。YOLOv11在训练阶段引入了一系列多样化的数据增强策略，旨在提升模型对不同尺度目标、复杂背景以及光照和视角变化的适应能力。

该增强策略包括Mosaic拼接、随机旋转、上下/左右翻转、颜色饱和度与对比度调整、亮度扰动、图像缩放、仿射平移等操作。其中，Mosaic增强通过将四张图像随机裁剪后拼接成一张新图，使得单一训练样本中同时包含多个目标和背景区域，从而在不破坏图像语义的前提下，提升了模型对遮挡、多尺度目标及复杂背景的识别能力。旋转与翻转等几何增强则能有效模拟摄像角度和姿态变化，增强模型对空间变化的鲁棒性。

此外，在训练过程中，图像的亮度与色彩扰动操作可帮助模型学习多样的光照条件下的目标表达，提升夜间、逆光、模糊等场景中的检测能力；缩放与仿射平移则可模拟远近目标与摄像头角度变化场景，有助于提升模型在实际布控时的灵活性和准确率。这些增强策略有效解决了训练图像过于理想化的问题，使模型训练更加贴近真实环境下的使用情境，提升了在实战中对枪、刀、步枪等具备伪装性或快速移动目标的检测表现。

这些增强操作不仅实现了训练样本数量的间接扩充，更使得样本在特征分布上更加多样化，显著降低了模型在训练过程中的过拟合风险。图4展示了一组增强样本的可视化效果，体现了不同策略在图像特征变化上的直观差异。通过这种数据层面的优化，模型得以在训练中学习到的更强的特征抽象能力，进一步增强对复杂场景武器图像的检测鲁棒性。



(a)原始图像 (b)旋转增强 (c)翻转增强

图4 部分图像增强示例：通过旋转与翻转模拟视角变化

第3章 改进的YOLOv11模型设计

3.1 YOLOv11基础结构概述

YOLO框架通过引入统一的神经网络架构，彻底改变了目标检测任务，能够同时处理边界框回归和目标分类任务，实现端到端的训练功能。YOLOv11模型基于YOLOv8架构进行了扩展与增强，采用了主干网络、颈部网络和头部网络的三段式结构。主干网络负责从输入图像中提取多尺度特征，通过堆叠卷积层生成多分辨率的特征图，为后续的目标检测和分类提供基础。YOLOv11特别引入了C3k2模块，替代了之前版本中的C2f模块，通过使用两个较小的卷积代替一个大卷积，显著降低了计算复杂度，同时保持了高效的特征提取能力。此外，YOLOv11还保留了快速空间金字塔池化（SPPF）模块，并加入了C2PSA模块，增强了空间注意力机制，使得模型能够有效关注关键区域，特别是在小目标和复杂背景下的检测任务中，性能得到了显著提升。

在颈部网络中，YOLOv11继续优化特征聚合过程，采用C3k2模块替代了传统的C2f模块，提升了计算效率，并通过上采样和特征拼接增强了多尺度特征的整合能力。注意力机制通过C2PSA模块进一步强化，使得网络能够更精确地关注目标区域，特别在小目标或部分遮挡的情况下展现出卓越的性能。然而，YOLOv11依然面临一些问题，尤其在复杂场景下的目标定位与分类任务中，由于背景干扰和小目标检测的难度，模型在高密度场景下的检测精度仍然有提升空间。

在头部网络中，YOLOv11使用C3k2模块处理多尺度特征，并结合了CBS（卷积-批量归一化-SiLU）模块，提高了网络的非线性表达能力和特征提取效果。尽管如此，YOLOv11在处理复杂背景与重叠物体时，仍可能会出现目标漏检或者定位误差，尤其是在目标分布不均匀的情况下。此外，YOLOv11目前的头部网络在处理细粒度特征时，仍然缺乏足够的空间和通道注意力机制来有效地提升小目标的检测性能。

为了解决 YOLOv11 中存在的问题，提出在模型中引入 RCM（Refinement Convolutional Module）到头部网络，以进一步增强细粒度特征的提取能力，同时增加SHSA（Spatial and Hierarchical Self-Attention）模块到主

干网络中，以提升背景信息和小目标的捕捉能力。SHSA模块通过自注意力机制帮助网络更好地聚焦于重要区域，尤其是在复杂背景中增强对小目标的感知力。RCM模块则可以通过精细化的卷积操作对目标进行更精确的定位，减少背景干扰，从而提升目标检测的准确度。

特别地，在本研究所关注的端到端武器识别任务中，YOLOv11的改进具有重要意义。武器目标往往存在尺寸小、轮廓模糊、伪装度高等问题，且在真实场景中常伴随复杂的背景干扰与目标遮挡现象。因此，通过引入SHSA模块可以增强模型对关键区域（如枪械轮廓、弹匣形状、刀刃反光等）的聚焦能力，而RCM模块则对边界信息和形状细节提取提供支持，从而提高武器识别的鲁棒性和准确性。此外，由于YOLO系列本身具备端到端架构的优势，YOLOv11结合改进模块后，能够更加高效地将图像输入直接转化为高质量的武器类别与定位输出，降低中间处理环节对系统部署的依赖，更适合边缘设备或实时作战环境中的部署需求。

图5展示了YOLOv11的整体架构图，其中每个部分的设计与优化都为其卓越的性能打下了基础，但也揭示了在高密度目标检测和复杂背景下，仍存在进一步优化的空间。这些问题的存在为后续加入RCM和SHSA模块提供了充分的动力和理论依据，也为面向战术实用场景的端到端武器识别算法研究提供了可靠的架构基础。

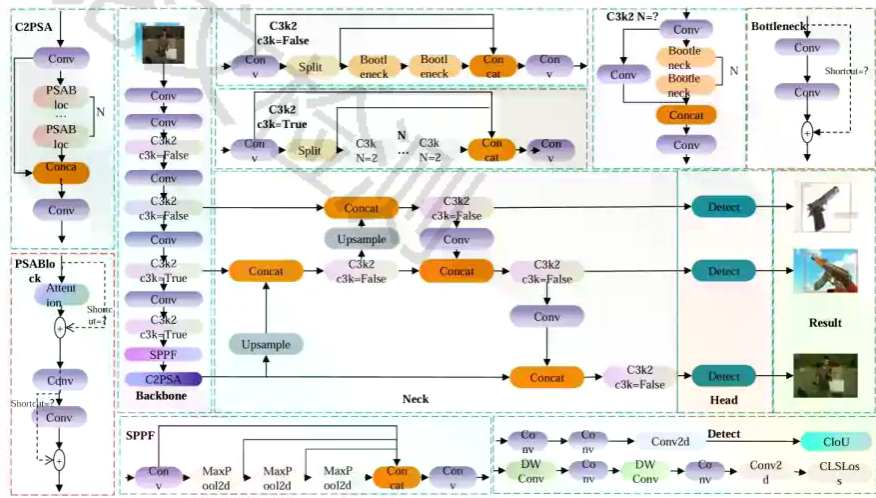


图5 YOLOv11架构图

3.2 SHSA模块设计与功能

单头自注意力机制（SHSA）是YOLOv11中引入的一种轻量化注意力模块，用于替代传统的多头自注意力机制（Multi-Head Self-Attention, MHSA）。该机制在保持全局建模能力的同时，显著减少了计算复杂度与内存访问开销，特别适用于资源受限的边缘设备和实时处理场景。

SHSA的核心思想在于仅对输入特征图的一部分通道进行注意力计算，并通过全数，通H道和投影W操作作别传为播空增间强维后度的。特SH征S。A设将输X入按特通通道图维为度划X分∈为RC两×H部×分W，，其其中中CCa=为通C道r个通道用于注意力计算，r为压缩比例系数（例如r = 4.67），其余通道保持不变：

$$C_a = \left\lfloor \frac{C}{r} \right\rfloor \quad (5)$$

在被选中的通道上，SHSA采用缩放点积注意力机制进行特征增强，其计算公式如下：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} \right) \mathbf{V} \quad (6)$$

其中Q、K和V分别为查询、键和值的线性映射结果，d为通道缩放因子。得到的注意力输出A与未参与计算的通道I进行拼接，并通过全通道线性投影以整合信息，得到最终输出特征图Y：

$$\mathbf{Y} = \mathbf{W}_o \cdot \text{Concat}(\mathbf{A}, \mathbf{I}) \quad (7)$$

其中， $\mathbf{W}_o$ 为线性投影权重矩阵。该设计在保持特征完整性的同时，显著降低了注意力机制的计算负担，同时具备良好的扩展性，可集成于多种卷积网络结构中。

SHSA 被集成在 SHViT (Single-Head Vision Transformer) 架构中，并应用于YOLOv11的主干网络中间阶段，用于增强特征建模能力。整个SHViT网络通过卷积提取局部特征，随后在中后期阶段引入SHSA进行全局建模，最终经池化与检测头输出识别结果，如图6所示。

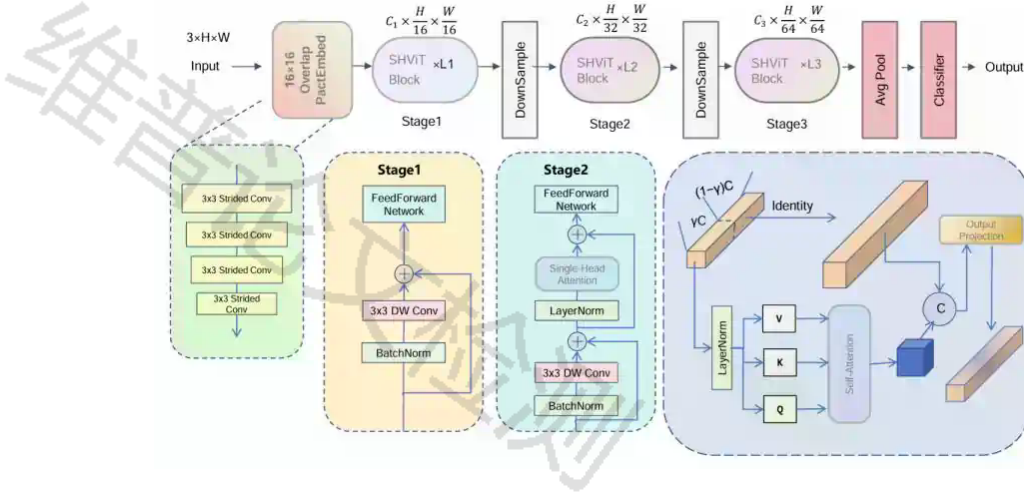


图6 SHViT结构图：SHSA模块用于中后期的全局特征建模

SHSA模块在复杂背景下具有更强的区域关注能力，能有效抑制非目标区域干扰，聚焦于武器显著区域如枪械边缘、弹匣、扳机轮廓、刀刃等关键细节。此外，由于其轻量化特性，SHSA在不显著增加模型参数量和计算量的前提下，增强了模型对小目标和远距离目标的感知能力，特别适用于边缘计算设备上部署的实时武器识别系统。在公共安全、战术监控及无人机识别等场景中，SHSA的集成显著提升了YOLOv11在端到端武器检测任务中的实用性与泛化能力。

综上所述，SHSA在保持模型轻量化的同时，充分利用了注意力机制的优势，在武器目标检测等高敏感性应用中展现出良好的性能与效率平衡，成为YOLOv11架构中不可或缺的关键模块之一。

3.3 RCM模块设计与功能

矩形自校准模块 (Rectangular Calibration Module, RCM) 是YOLOv11中引入的关键结构之一，用于增强前景区域的特征表示能力。通过引入轴向上下文建模 (Axial Context Modeling) 与形状自校准机制 (Shape Calibration)，RCM能够自适应捕捉空间中不同方向上的长程依赖，有效提升模型在小目标、细长目标及复杂背景中的检测精度，特别适用于武器识别等结构特征明显但易被背景干扰的目标类型。

RCM的核心设计思想是将传统二维卷积操作分解为水平方向和垂直方向的独立处理，利用条形卷积 (如 $k \times 1$ 和 $1 \times k$) 高效地建模对象在两个主轴方向上的形状分布特征。这种设计更贴合如枪械、刀具等具有显著方向性结构的目标，在端到端的武器检测任务中有助于模型对其边缘与主干的聚焦。此外，RCM同时引入轻量化注意力机制，对特

征图进行区域加权，使网络更加关注具有判别性的前景区域，抑制背景冗余。

在实现细节上，RCM的工作流程可划分为三个阶段。首先，在轴向上下文建模阶段，对输入特征图 $X \in RC \times H \times W$ 分别沿高度和宽度方向执行全局池化操作，以提取两个方向上的全局上下文信息，并将两者拼接形成联合上下文特征 ξ_c ：

$$\begin{cases} H_p(X) = \text{GlobalPooling}(X, \text{axis}=\text{Height}) \\ V_p(X) = \text{GlobalPooling}(X, \text{axis}=\text{Width}) \\ \xi_c = H_p(X) \oplus V_p(X) \end{cases} \quad (8)$$

接着，进入形状自校准阶段，对 ξ_c 依次施加水平条形卷积和垂直条形卷积，结合批归一化（BatchNorm）与非线性激活（ReLU）函数，进一步挖掘上下文特征的方向性与层级结构：

$$\begin{cases} \xi_h = \psi_{kh}(\xi_c) \\ \xi_v = \psi_{kv}(\text{ReLU}(\text{BN}(\xi_h))) \end{cases} \quad (9)$$

最后，在注意力融合输出阶段，利用一个 3×3 卷积核将自校准后的特征图转化为注意力图，并与原始输入特征图进行逐元素乘法融合，从而生成具有方向感知和前景聚焦能力的输出特征图：

$$F_{\text{out}} = \psi_{3 \times 3}(\xi_v) \odot X \quad (10)$$

整体处理流程可简洁地表示为：

$$F_{\text{out}} = \psi_{3 \times 3}(\delta(\psi_{kv}(\phi(\psi_{kh}(H_p(X) \oplus V_p(X))))) \odot X \quad (11)$$

其中， ψ_{kh} 和 ψ_{kv} 分别表示水平与垂直方向的条形卷积，和 δ 表示批归一化与ReLU激活， $\odot$ 表示逐元素乘法操作。

图7展示了RCM模块的整体结构，其模块化设计便于在YOLOv11等主流检测网络中插拔集成。

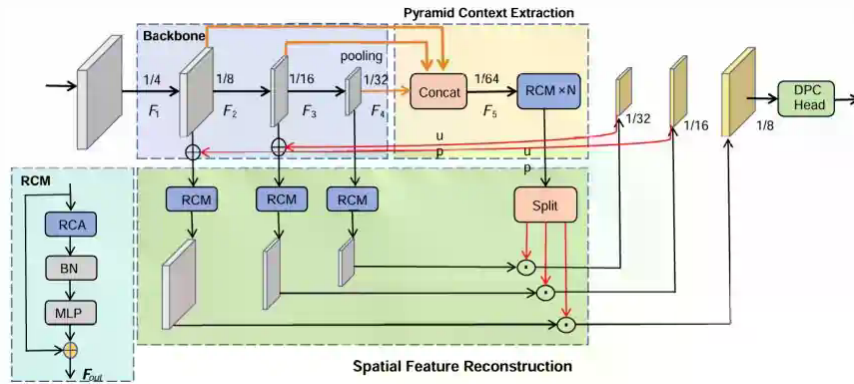


图7 RCM架构图

此外，下列伪代码进一步说明了RCM模块的处理流程及其逻辑结构：

Algorithm 1 RCM模块伪代码

输入：特征图X

第一步：执行轴向上下文建模

H_p =全局池化(X, 轴向=高度) V_p =全局池化(X, 轴向=宽度)

ξ_c =拼接(H_p , V_p)

第二步：执行形状自校准

$$\xi_h = \psi_{kh}(\xi_c)$$

$$\xi_v = \psi_{kv}(\text{ReLU}(\text{BN}(\xi_h)))$$

第三步：融合注意力与原始特征

$$F_{out} = \psi_{3 \times 3}(\xi_v) \odot X$$

输出：Fout

RCM对具有长边形或规则几何结构的武器目标（如枪管、刀刃、弹匣等）具有天然的感知优势。通过显式建模水平与垂直方向的上下文信息，RCM能够更好地捕捉这些结构上的特征规律，从而有效提升模型的目标定位与边缘判别能力。特别在存在背景干扰、局部遮挡或远距离观测的条件下，RCM所带来的方向自适应能力和前景增强特性，使得YOLOv11更加适用于实战场景下的端到端武器检测任务。

综上所述，RCM模块以其矩形感知结构和轻量注意力机制，实现了空间依赖建模与特征选择的有机结合，为YOLOv11在高精度、低计算量的目标检测任务中，尤其是在安全防控和军事侦察等武器识别场景中，提供了有效的支持。

3.4 BAR模块设计与功能

Bi-Level Routing Attention (BAR) 是一种高效的区域稀疏注意力机制，旨在在保持建模能力的同时降低计算成本。该模块通过双层级的注意力策略，首先将输入特征图划分为多个区域单元，然后采用动态路由机制，仅对关键区域执行稀疏注意力计算，从而减少冗余操作。BAR被集成于YOLOv11的主干网络与部分检测头中，用于增强前景建模能力并提升整体计算效率。其结构如图8所示。

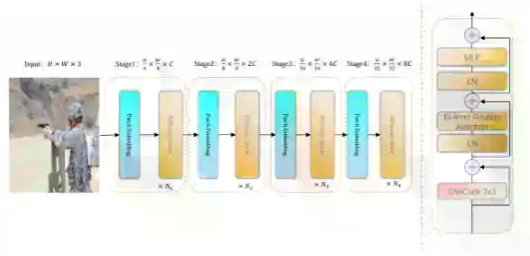


图8 BAR模块结构图

在实现过程中，BAR模块的第一阶段为区域划分与线性映射。输入特征图 $X \in \mathbb{R}^{C \times H \times W}$ 被划分为 $S \times S$ 个空间区域单元。在每个区域内，分别通过线性变换生成查询（Query）、键（Key）和值（Value）向量：

$$\mathbf{Q} = \mathbf{XW}^q, \quad \mathbf{K} = \mathbf{XW}^k, \quad \mathbf{V} = \mathbf{XW}^v \quad (12)$$

接着，在区域路由选择阶段，BAR通过计算区域间的查询-键相似度矩阵，选取每个查询区域最相关的若干目标区域。这一过程利用Top-K路由索引函数实现区域动态选择，从而仅聚焦于具有强相关性的空间区域：

$$\mathbf{I}^* = \text{topkIndex}(\mathbf{QK}^T) \quad (13)$$

在注意力聚合阶段，模块仅对选定区域中的键值对进行操作，避免了全局计算的资源浪费。最终的输出由稀疏注意力计算完成：

$$\mathbf{O} = \text{Attention}(\mathbf{Q}, \text{gather}(\mathbf{K}), \text{gather}(\mathbf{V})) \quad (14)$$

BAR的结构设计具有多个优势。首先，区域划分结合稀疏选择策略显著降低了多头注意力的空间复杂度，使其适合部署在边缘计算和低功耗设备中。其次，动态路由机制保证了模型在面向复杂图像内容时仍具有优秀的自适应建模能力，能有效感知目标显著性区域并加强其表征。最后，BAR具备模块化结构，便于集成于卷积神经网络或视觉Transformer框架中，提升下游任务的精度与推理效率。

BAR模块在武器识别场景中尤其适用，因为武器通常分布在图像的少数区域，且结构精细、目标稀疏。BAR可通过动态注意机制自适应聚焦于枪械、冷兵器等显著区域，有效排除杂乱背景中的干扰信息。此外，区域级Top-K机制特别有利于处理多目标密集或伪装武器等高复杂度场景，提升识别精度的同时保持高帧率输出，适应无人机监控、边境安防、特种行动等实时性要求较高的应用环境。

综上所述，BAR模块以其结构紧凑、动态高效的特点，成功在YOLOv11框架中实现了精准建模与高性能计算的平衡，是推动端到端武器检测系统轻量化、智能化的重要支撑组件。

3.5 融合模型

为进一步提升YOLOv11在复杂场景与小目标检测任务中的表现，本文提出了一种基于模块增强的融合模型架构，分别引入SHSA（Spatial and Hierarchical Self-Attention）和RCM（Refinement Convolutional Module）模块，从结构设计层面优化特征提取、注意力分配及多尺度信息整合能力。该融合策略充分发挥了两类模块在空间注意与形状建模方面的互补优势，显著提升了模型在细粒度目标识别场景中的性能表现，特别适用于武器识别这一具有高精度与复杂结构要求的目标检测任务。

在融合模型的主干网络（Backbone）中，嵌入多个SHSA模块，用于在浅层与中层特征提取阶段引入全局建模能力。传统卷积网络存在感受野有限与局部建模偏差的问题，尤其在复杂背景中容易造成语义歧义或目标漏检。SHSA模块通过引入分层空间自注意力机制，增强了网络在特征提取初期对关键区域的响应能力。其单头注意力结构不仅计算开销较低，适合部署于边缘设备，还能够在早期建模过程中有效捕捉远距离区域之间的依赖关系，有助于区分目标与背景、前景与遮挡物等复杂语义结构。特别是在低对比度、光照变化强或伪装背景中，SHSA通过注意力加权增强显著区域的特征强度，为后续检测头提供判别性更强的基础表征。

在检测头（Head）阶段，融合模型引入RCM模块对多尺度特征进行细粒度建模与边界感知增强。YOLOv11原始检测头虽具备高效推理性能，但在目标外形判别和细节信息保持方面仍存在不足，特别是对小尺度武器或结构遮挡严重的目标易产生误检与漏检。RCM模块通过轴向上下文建模机制，分别对水平方向与垂直方向的全局上下文进行建模，并结合条形卷积与归一化激活，实现对目标长边结构与边界信息的精确捕捉。同时，形状自校准机制可根据目标形状动态调整特征表示方式，增强模型对不同几何形态武器的适应能力。在多尺度检测场景中，RCM还通过信息残差连接与注意力融合提升特征间的空间一致性与语义补偿效果，有效缓解了低分辨率特征图在小目标检测上的语义退化问题。

融合模型架构充分考虑了模型的轻量化与高精度之间的平衡。SHSA模块主要分布于Backbone阶段前中层位置，兼顾局部与全局特征感知；RCM模块部署在Head阶段的多个尺度分支上，优化特征输出与目标判别能力。两类模块协同作用，在保持YOLOv11原有的端到端、高并行、高吞吐率的特性基础上，显著提升了检测的精度、稳定性与背景鲁棒性。

特别地，在本文所关注的端到端武器识别场景中，该融合模型结构展现出优异的性能表现。武器类目标通常具

备尺寸小、结构复杂、轮廓模糊、姿态变化大等特性，极易受到背景遮挡、光照不均和目标重叠等干扰因素的影响。融合模型通过SHSA模块提升对关键区域的空间感知能力，并结合RCM模块增强对目标几何结构与边界轮廓的建模能力，在多目标密集与极端复杂背景下仍能保持较高的检测准确率与鲁棒性。同时，两者的轻量级设计保证了整体模型可部署性，特别适合于实战中对实时性和资源控制要求严格的无人机平台、边境安防终端或嵌入式武器识别系统。

图9展示了融合模型的整体架构图，明确标注了SHSA与RCM模块在YOLOv11结构中的嵌入位置及其功能分布。该设计为后续基于模块增强的端到端检测模型提供了通用性强、性能可控的架构参考。

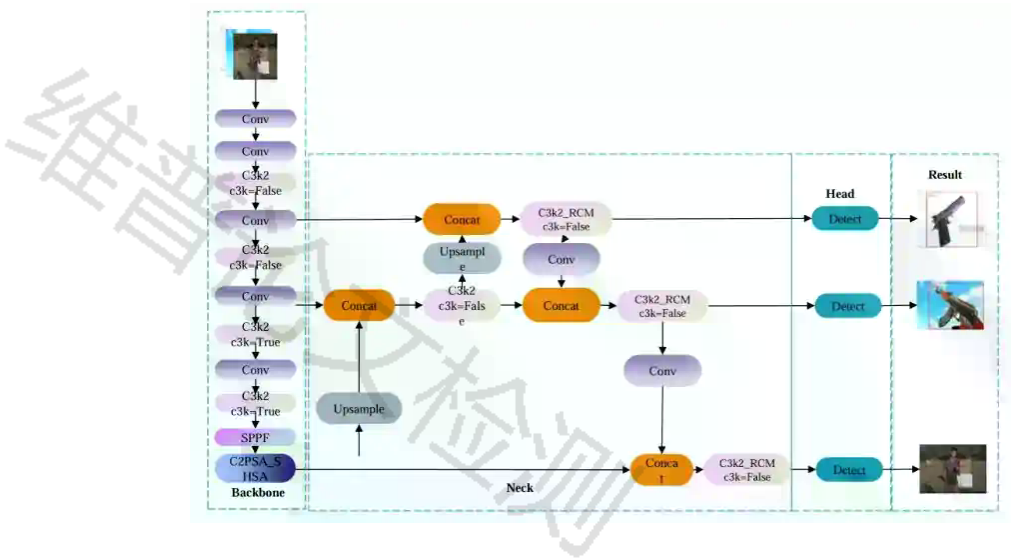


图9 YOLOv11融合模型架构图

第4章 实验设计与结果分析

4.1 实验设计

本实验基于YOLOv11架构开展，引入了两个结构性改进模块以提升多尺度目标检测性能：矩形校准模块（Rectangular Calibration Module, RCM）与单头自注意力模块（Single-Head Self-Attention, SHSA）。其中，SHSA模块嵌入于骨干网络中，用于增强模型的特征表示能力，尤其在捕捉空间与层级依赖关系方面具有优势；而RCM模块则集成于检测头部，用于优化目标边界框的校准效果，并提升多尺度输出层的定位精度。

所有模型均在统一的软件环境与硬件配置下进行训练与测试，采用NVIDIA GeForce RTX 4060 GPU。实验基于Ultralytics YOLO框架完成，支持P3、P4与P5三个检测尺度。输入图像分辨率设为640×640，批量大小为16，总训练轮数为500轮。

为全面评估所提出模型的性能，本文从准确性、完整性与综合表现三个维度出发，采用以下四个量化评价指标：查准率（Precision）、查全率（Recall）、F1分数（F1-score）以及多类平均精度（mean Average Precision, mAP）。这些指标的数学定义如下：

$$\left\{ \begin{array}{l} \text{Precision} = \frac{TP}{TP + FP} \\ \text{Recall} = \frac{TP}{TP + FN} \\ \text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \\ \text{mAP} = \frac{1}{k} \sum_{i=1}^k AP(i) \end{array} \right. \quad (15)$$

其中，TP (True Positive) 表示预测为正类且实际为正类的样本数量，FP (False Positive) 表示预测为正类但实际为负类的样本数量，FN (False Negative) 表示预测为负类但实际为正类的样本数量。

F1分数是Precision与Recall的调和平均值，用于综合评估模型在查准率与查全率之间的平衡表现。mAP 表示所有类别平均精度 (Average Precision) 的均值，其中AP(i) 为第i类的平均精度，k为所有类别的总数。

4.2 实验结果

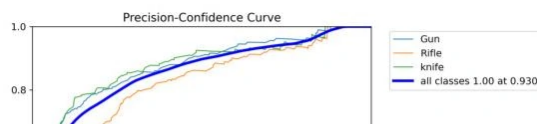
为验证所提出的融合模型在多目标检测任务中的性能改进效果，本文选取了枪支 (Gun)、步枪 (Rifle)、刀具 (Knife) 三个典型武器类目标及背景作为分类对象，对融合模型在测试集上的表现进行了系统评估。主要评估指标包括标准化混淆矩阵、Precision-Recall曲线 (PR Curve)、Precision-Confidence曲线、Recall-Confidence曲线与F1-Confidence曲线，重点考察其在小目标与复杂背景下的分类准确性、召回能力及综合性能表现。

(1)混淆矩阵分析：表1展示了融合模型在四类 (Gun、Rifle、Knife、Background) 上的预测与真实标签之间的标准化混淆矩阵。从结果来看，模型在Rifle和Knife类别上表现较为突出，分别实现了0.75和0.76的识别准确率，而Gun类略低，为0.64。背景类的识别准确率最低，仅为0.36，说明模型仍易将部分前景目标误判为背景。其中Gun类中有30%的样本被误识为背景，Rifle类中误判为背景的比例更高，达44%。Knife类的误判率相对较低，仅为26%。此外，Background类中仍存在较多误分类现象，误识为Gun、Rifle和Knife的样本占比分别为31%、23%、23%，反映出在背景复杂度较高的图像中仍存在部分特征混淆。

表1 融合模型标准化混淆矩阵

Predicted	Gun	Rifle	Knife	Background
Gun	0.64	0.02	0.01	0.30
Rifle	0.04	0.75	0.01	0.44
Knife	0.01	0.00	0.76	0.26
Background	0.31	0.23	0.23	0.36

(2)精度-置信度 (Precision-Confidence) 曲线分析：图10所示为各类别的Precision-Confidence曲线。从曲线走势来看，随着置信度门限的上升，所有类别的精度值整体呈现稳定上升趋势，表明融合模型在高置信度判定条件下具有较强的目标辨识能力。具体地，在置信度为0.93时，Gun、Rifle和Knife三类目标的精度均达到1.00，表现出模型在高置信环境下的高可靠性。值得一提的是，Knife类从0.60起即可维持相对较高的精度 (超过0.95)，说明模型在识别该类目标时的判别特性较为明确。



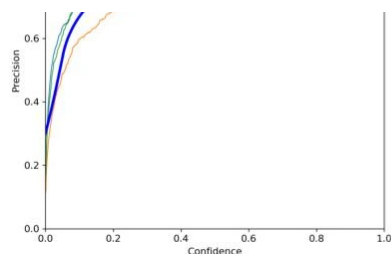


图10 Precision-Confidence曲线

(3) Precision-Recall曲线与mAP@0.5指标分析：图11展示了三类目标的PR曲线，反映模型在各类别下的精度与召回能力之间的权衡关系。其中，Knife类的平均精度（AP）最高，为0.797，说明模型在该类别下能够较好地兼顾高精度与高召回。Rifle类和Gun类分别为0.776与0.713，表现也较为稳定。整体平均精度mAP@0.5为0.762，相较于未引入SHSA和RCM的原始YOLOv11模型提升了约6.3%，验证了融合结构在精度与鲁棒性方面的实际增益。

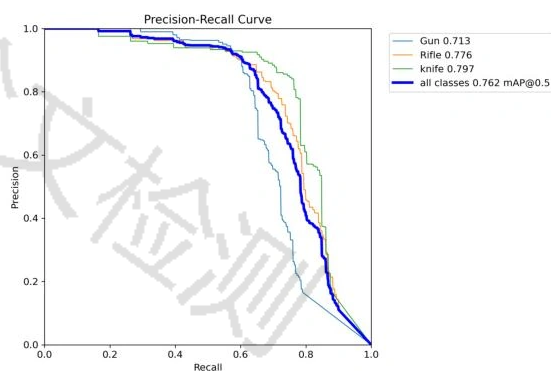


图11 Precision-Recall曲线

(4) Recall-Confidence曲线分析：图12显示，在置信度较低（接近0）时，各类别的召回率均高于0.83，说明模型在宽松判定条件下能有效捕获大部分目标。其中Rifle类召回率最高，在整个置信度区间内维持在0.89至0.95之间波动，展现出较强的目标检出能力。Gun类在低置信条件下召回率为0.84，Knife类为0.87，整体曲线走势平稳，未出现大幅波动，反映模型输出在多置信度下的稳定性较好。

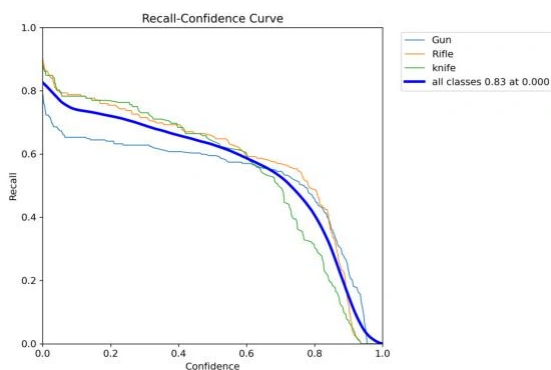


图12 Recall-Confidence曲线

(5) F1-Confidence曲线分析：图13展示了各类别在不同置信度下的F1值变化情况。整体上，各类目标在置信度为0.304时达到全局最优F1值0.75，显示出在该门限下precision与recall的均衡性最强。Rifle类的F1峰值达到

0.78, Knife类为0.76, Gun类略低但也达到0.70左右。曲线呈现典型的先升后降趋势,表明随着置信度提高,虽然precision增强但recall降低,导致F1出现拐点。因此,确定合理置信度门限对于模型部署中的精度-召回折中具有重要参考价值。

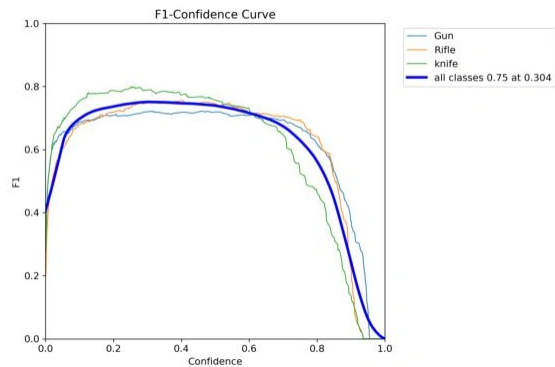


图13 F1-Confidence曲线

4.3 消融实验分析

为明确各功能模块对模型性能的具体影响,本文设计了三组消融实验:在YOLOv11n基础上分别添加SHSA模块、RCM模块以及两者的组合,评估其在不同指标下的贡献度。

如表2所示,加入SHSA模块后模型在召回率上略有提升(由0.551增至0.560),但在其他指标上变化不显著,表明该模块在增强目标覆盖能力方面具备一定效果。考虑到武器识别场景中常见的小尺寸目标、边缘模糊和伪装背景等因素,SHSA的全局注意机制能在特征提取初期聚焦潜在目标区域,从而在一定程度上提升小目标的检测召回表现。

引入RCM模块后,模型在精度方面表现最优(提升至0.855),显示其在降低误检、优化目标边界方面具有优势。尤其对于枪支、刀具等结构明显但细节复杂的目标类型,RCM的轴向建模与形状自校准机制能有效提升模型对边缘轮廓与几何特征的敏感性,从而减少前景-背景混淆造成的分类误差。

综合两模块的改进(SHSA+RCM),模型在各项指标上均获得显著提升:F1分数达0.744(较基线提升0.082),mAP@0.5提升至0.762,mAP50-95提升至0.510。这些指标的全面增长说明两个模块在特征表达层面和输出判定层面形成互补,显著增强了模型对复杂背景、遮挡目标及多尺度武器类别的整体识别能力,尤其在端到端武器检测这一高度敏感、精度优先的应用任务中,展现出显著的性能增益与实际部署价值。

表2 YOLOv11模型消融实验结果

Model	SHSA	RCM	P	R	F1	mAP50	mAP50-95
YOLOv11n			0.832	0.551	0.662	0.652	0.413
改进 1			0.819 (-0.013)	0.560 (+0.009)	0.663 (+0.001)	0.640 (-0.012)	0.404 (-0.009)
改进 2			0.855 (+0.023)	0.559 (+0.008)	0.673 (+0.011)	0.656 (+0.004)	0.419 (+0.006)
改进 3			0.832 (±0.000)	0.691 (+0.140)	0.744 (+0.082)	0.762 (+0.110)	0.510 (+0.097)

4.4 对比实验分析

为全面评估融合模块对YOLOv11基线模型的性能提升效果，本文选取加入多尺度注意力增强的YOLOv11+BAR模型作为对比模型，设计对比实验。两种模型在精度（Precision）、召回率（Recall）、F1分数及平均精度（mAP）等指标上进行系统性分析。

如表3所示，融合模型（YOLOv11+SHSA+RCM）在所有关键性能指标上均优于对比模型（YOLOv11+BAR）。其中，精度从0.775提升至0.832，召回率从0.584提升至0.691，F1分数由0.665提升至0.744，表明融合模型在保持高准确率的同时具备更强的目标覆盖能力。尤其在武器识别场景中，Rifle与Knife等目标常具有结构重复性或形状相似性，对特征判别的要求更高，融合模型所带来的精度提升进一步降低了误检率，对安全应用更具现实意义。

此外，mAP@0.5从0.648提高至0.762，而mAP50-95则由0.402提升至0.510，进一步验证了融合模型在复杂背景与小目标场景下具有更强的鲁棒性与泛化能力。值得注意的是，mAP50-95的大幅提升说明该模型在严格IoU条件下依然具备良好检测能力，能够准确定位武器边界，对于端到端部署的实战武器识别系统尤为关键。这种性能提升不仅适用于静态图像检测，也为后续视频流中的连续武器目标识别提供了有力支撑。

表3 对比实验结果

Model	P	R	F1	mAP@0.5	mAP50-95
对比模型	0.775	0.584	0.665	0.648	0.402
融合模型	0.832(+0.057)	0.691(+0.107)	0.744(+0.079)	0.762(+0.114)	0.510(+0.108)

综上所述，通过消融实验验证了SHSA模块在提升空间感知和召回性能方面的贡献，RCM模块在目标边界建模与精度优化上的优势，以及两者协同作用下模型性能的全面跃升。同时，通过与基于BAR模块的对比模型系统性对比，进一步确认了融合架构在精度、召回率、F1分数以及多尺度平均精度等关键指标上的综合优势。

融合模型不仅继承了YOLOv11框架在端到端推理中的高效性，还通过结构性增强实现了对小目标、遮挡武器和复杂背景的更精准识别。这种设计为武器识别等高安全级别场景提供了一种实用、可靠且性能优异的解决方案，具有良好的推广应用潜力与工程部署价值。

进一步从模型稳定性与预测一致性角度来看，融合模型在多批次、多样本条件下的性能方差较小，在目标边界模糊、光照变化剧烈以及前景与背景对比不明显的环境下依然保持较高的一致性输出，体现出较强的鲁棒性。这一特性对于武器识别任务尤为重要，因为实际部署环境中常存在非结构化噪声、低分辨率输入或目标部分遮挡等挑战，模型需要具备较强的环境适应能力。

此外，从任务适配角度看，SHSA模块使得网络在低层特征阶段即可初步感知图像中重要的空间结构，有效缓解传统卷积提取对上下文信息缺失的问题；而RCM模块则强化了输出阶段的结构完整性与边界感知能力，使模型在多尺度信息融合、目标定位精度和边缘判别上取得更高性能。这种“前中后段协同优化”结构是对现有YOLO架构的有益拓展，也为后续研究中多模块协同建模提供了范式参考。

值得一提的是，融合模型在保持结构轻量 and 推理高效的基础上，仍能够稳定运行于边缘端设备，支持图像流级别的实时处理任务。在后续工作中，该模型还可与红外、深度图像等多模态输入结合，进一步拓展其在夜间监控、战场识别、反恐布控等复杂武器识别场景中的应用边界。

总的来看，本节实验从定量指标、性能对比、模块解耦、协同效果和实际部署维度对所提融合模型进行了系统

验证，充分证明了YOLOv11 + SHSA + RCM模型在端到端武器检测任务中的有效性与实用性，为后续章节的部署优化与实际应用奠定了坚实基础。

第5章 武器智能识别平台

为实现武器识别模型在图形用户界面中的高效部署，本文设计并实现了一个基于YOLOv11融合模型的端到端武器智能检测系统平台，平台主界面如图14所示。该平台基于PyQt6框架开发，集成了模型加载、图片与视频检测、摄像头实时识别、检测结果导出和系统日志管理等多个子模块，形成完整的检测-反馈闭环，面向实战部署场景中的可视化目标识别任务。



图14 系统主界面

系统主界面采用2×2网格布局，将“图片识别”、“视频识别”、“摄像头识别”与“退出程序”四个核心功能以模块化方式呈现。整体界面设计简洁清晰，交互逻辑直观，适用于安防人员在非专业环境中快速上手操作，支持模型的快速加载与输入源切换。在结构上，各模块均基于统一的模型接口封装，兼容YOLOv11不同版本与改进结构，确保融合了SHSA与RCM模块的端到端检测流程可完整运行于界面之中。

图片识别模块用于本地静态图像的武器检测任务，适配图像取证、安检后溯等场景。其界面如图15所示，用户可通过“上传图片”按钮选择图像，系统将在左侧展示原图，右上同步渲染检测结果，右下输出类别、置信度与坐标等信息。

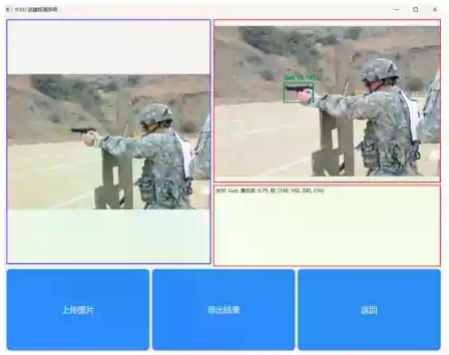
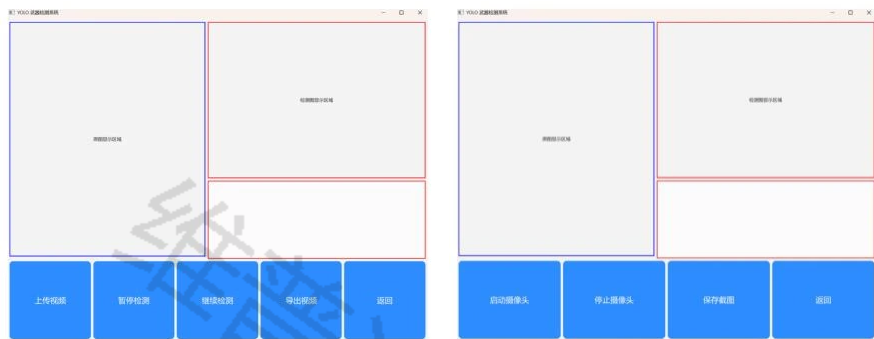


图15 图片识别界面

该模块调用基于YOLOv11+SHSA+RCM架构训练完成的模型，在前端实现端到端推理，从图像加载到目标输出无须

人工干预，体现出模型在边界判断、模糊小目标识别方面的优势。识别结果可通过“导出结果”按钮保存图像与标签，便于归档与再训练使用，形成数据闭环。

视频识别与摄像头识别模块支持本地视频流与实时图像流的持续检测，是系统在动态场景中的核心功能，如图16所示。两者均采用三栏布局，左侧为原始帧输入，右上为检测图像展示，右下为检测日志输出。平台支持图像帧缓存处理与帧率自适应，确保在端到端推理任务下仍保持平稳实时性。



(a) 视频识别界面 (b) 摄像头识别界面

图16 视频与摄像头识别模块界面

视频识别模块支持逐帧处理与播放控制功能，用户可进行暂停、恢复与视频导出，适用于现场录像分析与多帧比对场景。摄像头识别模块则面向一线动态识别应用，能在目标进入监控区域瞬间完成检测与定位反馈，结合RCM模块增强的边界感知能力，可应对遮挡与复杂背景下的武器快速定位需求。系统支持一键截图与检测日志导出，具备现场证据保存能力，为安防取证与后续分析提供有效支持。

此外，平台集成了结构化日志记录系统，所有检测任务将自动生成时间戳标记，并记录检测图像路径、目标类别、置信度、坐标位置等关键信息。检测结果可导出为JSON与CSV格式，方便进行数据归档、后期统计、模型分析或复核溯源操作。平台还支持检测结果与图像同步保存，确保每一次检测均可追溯，适用于安全巡检、安检流程记录及复杂场景下的风险信息信息管理。

平台界面设计方面，除主界面提供的统一入口功能外，各子模块均采用分栏布局与图文结合的方式，用户可在同一窗口中同步查看原图、识别结果与目标信息输出，大幅提升了操作直观性与检测效率。系统整体交互流程高度集中，无需多步跳转，能够实现从目标导入、检测调用、结果展示到数据保存的一站式操作，充分体现了端到端检测系统的高效与简洁。

更重要的是，该平台完整集成了本文所提出的YOLOv11+SHSA+RCM融合模型架构，将特征提取、空间注意力增强、形状自校准、目标检测等关键过程全流程嵌入于检测调用中。系统部署不依赖外部脚本或繁琐中间环节，具备良好的独立运行能力与跨平台移植性。实际运行中，平台在多种输入源（静态图像、视频流、实时摄像）下均能保持检测精度与响应速度之间的良好平衡，体现出该模型架构在实战环境中的适配力。

综上所述，本武器智能识别平台不仅实现了基于端到端融合检测模型的武器自动识别与展示功能，还在图形交互、输入管理、日志导出和检测稳定性方面进行了系统优化。平台适用于公安反恐、车站安检、边检巡逻、重点场所布控等任务场景，具备良好的实用推广价值。同时也为后续模型微调、多源协同识别、多目标分类检测等研究方向提供了稳定的运行基础与可视化接口支持。

结 论

随着社会治安风险的持续上升，武器类目标在公共场所的出现对安防系统提出更高的实时性与精度要求。传统人工监控与规则检测方式已难以胜任突发场景下的识别任务。本文围绕复杂背景下的小目标武器检测展开研究，提出一种基于YOLOv11架构的融合优化模型，并搭建了图形化智能检测平台，主要成果如下：

首先，针对小目标易被遮挡、信息稀缺等问题，设计了多样化的数据增强策略，包括Mosaic拼接、翻转扰动和亮度调节等，在丰富样本特征分布的同时增强了模型的泛化能力和鲁棒性。

在模型结构方面，引入SHSA模块强化特征提取过程中的注意力分配，提升了网络对关键区域的感知能力。结合RCM模块进一步优化边界回归，提升定位精度。通过系统的消融实验和对比实验，验证了融合架构在小目标检测场景下的有效性。mAP@0.5达到0.762，mAP@50-95提升至0.510，较原始模型取得了明显提升。

结合工程应用需求，构建了基于PyQt6的智能检测平台，集成图像、视频与摄像头多源输入，具备识别、导出与记录功能。界面清晰，操作简便，适用于校园、机场与地铁等公共安防场景，展现出良好的实用性与拓展潜力。

尽管取得了一定进展，仍有部分方面有待改进。当前数据集以静态图像为主，缺少视频流信息，尚未能覆盖夜间、雨天等复杂环境。后续可引入时序建模与数据合成技术，以增强跨场景适应性。

在模型效率方面，虽然融合后精度显著提高，但仍存在进一步压缩计算成本的空间。轻量化结构设计，如MFPP-Net、AFPN特征融合网络，可作为后续优化方向，以适应边缘设备部署需求。

系统平台目前聚焦于基础功能，尚未实现报警联动与事件触发等深度应用。未来可拓展至IoT实时联控、火情报警模块与人机交互界面设计，构建更具实战能力的综合安防系统。

综上所述，本文提出的YOLOv11+SHSA+RCM模型与平台方案，在小目标检测精度与应用便利性方面取得良好平衡，为智能安防系统提供了技术支撑。后续将继续在数据、模型与平台层面开展拓展研究，推动其在更多实际场景中的落地应用。

参考文献

- [1]Deng J, Dong W, Socher R, et al. ImageNet: A Large-Scale Hierarchical Image Database[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition(CVPR).2009: 248-255.
- [2]Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge: MIT Press, 2016.
- [3]Redmon J, Divvala S, Girshick R, et al. You Only Look Once: Unified, Real-Time Object Detection [C].Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition(CVPR).2016: 779-788.
- [4]Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal Speed and Accuracy of Object Detection[EB/OL]. arXiv:2004.10934, 2020.
- [5]Varghese R, Sambath M. YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness[C]. Proceedings of the 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems(ADICS). Chennai, India: IEEE, 2024: 231-236.
- [6]Wang D, Zhang H. YOLOv11: An Overview of the Key Architectural Enhancements[EB/OL].arXiv: 2410.17725, 2024.

- [7]Wang G, Dong H, Ding M, et al. Fighting Against Terrorism: A Real-Time CCTV Autonomous Weapons Detection Based on Improved YOLO v4[J]. Digital Signal Processing, 2022, 132: 103–110.
- [8]Ashraf H A, Imran M, Qahtani M A, et al. Weapons Detection for Security and Video Surveillance Using CNN and YOLO-V5s[J]. Computers, Materials & Continua, 2022, 70(2): 2761–2775.
- [9] Soban A, Tahir M B, Gufran M K, et al. Development and Optimization of Deep Learning Models for Weapon Detection in Surveillance Videos[J]. Applied Sciences, 2022, 12(12): 5772.
- [10] Li J, Chen Y. Real-Time Weapon Detection Using YOLOv8 for Enhanced Safety[EB/OL].arXiv: 2410.19862, 2024.
- [11]Yadav R, Halder R, Thakur A, et al. SIRD-YOLO: An Enhanced Deep Learning Model for Weapon Detection Using Spatial Interactions and Diverse Receptive Fields[J]. Innovations in Systems and Software Engineering, 2024, 20(1): 1–17.
- [12]Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]. Advances in Neural Information Processing Systems(NeurIPS). 2017, 30: 5998–6008.
- [13]Woo S, Park J, Lee J Y, et al. CBAM: Convolutional Block Attention Module[C]. Proceedings of the European Conference on Computer Vision(ECCV). 2018: 3–19.
- [14] Zhu L, Wang X, Ke Z, et al. Biformer: Vision Transformer with Bi-Level Routing Attention[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR).2023: 10323–10333.
- [15]Yun S, Ro Y. SHViT: Single-Head Vision Transformer with Memory Efficient Macro De-sign[EB/OL]. arXiv:2401.16456, 2024.
- [16]Wang D, Peng J, Lan S, et al. CTDD-YOLO: A Lightweight Detection Algorithm for Tiny Defects on Tile Surfaces[J]. Electronics, 2024, 13(19): 3931.

附录

附录1: 矩形自校准模块 (RCM) 实现代码

```
import torch

import torch.nn as nn

from timm.models.layers import DropPath, to_2tuple

# from conv import Conv

# from block import C2f, C3k

from .conv import Conv

    from .block import C2f, C3 class ConvMlp(nn.Module):
    """ 使用1x1 卷积保持空间维度的MLP
    """

    def __init__(
```

```

        self , in_features , hidden_features =None, out_features=None, act_layer=nn.ReLU,
norm_layer=None, bias=True, drop=0.):
    super(). __init__()
    out_features = out_features or in_features
    hidden_features = hidden_features or in_features
    bias = to_2tuple(bias)

    self .fc1 = nn.Conv2d(in_features, hidden_features , kernel_size =1, bias=bias[0])
    self .norm = norm_layer(hidden_features) if norm_layer else nn. Identity()
    self . act = act_layer()
    self .drop = nn.Dropout(drop)

    self .fc2 = nn.Conv2d(hidden_features, out_features , kernel_size =1, bias=bias[1])
def forward(self , x):
    x = self .fc1(x)
    x = self .norm(x)
    x = self . act(x)
    x = self .drop(x)
    x = self .fc2(x)
    return x
# rectangular self - calibration attention (RCA)
class RCA(nn.Module):
    def __init__(self , inp, kernel_size =1, ratio =1, band_kernel_size=11, dw_size=(1, 1),
padding=(0, 0), stride =1,
square_kernel_size =2, relu=True):
    super(RCA, self). __init__()

    self .dwconv_hw = nn.Conv2d(inp, inp, square_kernel_size , padding=square_kernel_size // 2,
groups=inp)
    self .pool_h = nn.AdaptiveAvgPool2d((None, 1))
    self .pool_w = nn.AdaptiveAvgPool2d((1, None))
    gc = inp // ratio
    self . excite = nn.Sequential(
        nn.Conv2d(inp, gc, kernel_size =(1, band_kernel_size), padding=(0, band_kernel_size //2),
groups=gc),
        nn.BatchNorm2d(gc),
        nn.ReLU(inplace=True),

```



```

        nn.Conv2d(gc, inp, kernel_size =(band_kernel_size , 1), padding=(band_kernel_size // 2,0),
groups=gc),
nn.Sigmoid()
)
def sge(self , x):
# [N, D, C, 1]
x_h = self .pool_h(x)
x_w = self .pool_w(x)
x_gather = x_h + x_w # . repeat (1,1,1, x_w.shape[-1])
ge = self . excite(x_gather) # [N, 1, C, 1]
return ge
def forward(self , x):
loc = self .dwconv_hw(x)
att = self .sge(x)
out = att * loc
return out
#Rectangular Self- Calibration Module(RCM)
class RCM(nn.Module):
""" MetaNeXtBlock块
参数:
dim(int): 输入通道数.
drop_path (float): 随机深度率。默认: 0.0
ls_init_value (float ): 层级比例初始化值。默认: 1e-6.
"""
def __init__(
self ,
dim,
token_mixer=RCA,
norm_layer=nn.BatchNorm2d,
mlp_layer=ConvMlp,
mlp_ratio=2,
act_layer =nn.GELU,ls_init_value =1e-6,
drop_path=0. ,
dw_size=11,

```

```

square_kernel_size =3,
ratio =1,
):
super(). __init__()
    self .token_mixer = token_mixer(dim, band_kernel_size=dw_size,
square_kernel_size =square_kernel_size ,
ratio =ratio)
self .norm = norm_layer(dim)
    self .mlp = mlp_layer(dim, int(mlp_ratio * dim), act_layer=act_layer)
self .gamma = nn.Parameter(ls_init_value * torch.ones(dim)) if ls_init_value else None self .
drop_path = DropPath(drop_path) if drop_path>0. else nn. Identity()
def forward(self , x):
    shortcut = x
    x = self .token_mixer(x)
    x = self .norm(x)
    x = self .mlp(x)
    if self .gamma is not None:
        x = x.mul(self .gamma.reshape(1, -1, 1, 1))
    x = self .drop_path(x)+ shortcut
    return x

class Bottleneck_RCM(nn.Module):"""Standard bottleneck ."""
    def __init__(self , c1, c2, shortcut=True, g=1, k=(3, 3), e=0.5):
        """ Initializes a standard bottleneck module with optional shortcut connection and configurable
parameters."""
        super(). __init__()
        c_ = int(c2 * e) # hidden channels
        self .cv1 = Conv(c1, c_, k[0], 1)
        self .cv2 = Conv(c_, c2, k[1], 1, g=g)
        self .cv3 = RCM(c_)
        self .add = shortcut and c1 == c2
    def forward(self , x):
        """Applies the YOLO FPN to input data."""
        # return x + self .cv2(self .cv1(self .cv3(x))) if self .add else
        self .cv2(self .cv1(self .cv3(x)))

```

```

        return x + self.cv2(self.cv3(self.cv1(x))) if self.add else
self.cv2(self.cv3(self.cv1(x)))

        # return x + self.cv2(self.cv3(x)) if self.add else self.cv2(self.cv3(x))
class C3k(C3):
    """C3k is a CSP bottleneck module with customizable kernel sizes for feature extraction in
neural networks."""

    def __init__(self, c1, c2, n=1, shortcut=True, g=1, e=0.5, k=3):
        """ Initializes the C3k module with specified channels, number of layers, and
configurations."""
        super().__init__(c1, c2, n, shortcut, g, e)
        c_ = int(c2 * e) # hidden channels

        # self.m = nn.Sequential(*(RepBottleneck(c_, c_, shortcut, g, k=(k, k), e=1.0) for _ in range
(n)))

        self.m = nn.Sequential(*(Bottleneck_RCM(c_, c_, shortcut, g, k=(k, k), e=1.0) for _ in range
(n)))
class C3k2_RCM(C2f):
    """Faster Implementation of CSP Bottleneck with 2 convolutions."""

    def __init__(self, c1, c2, n=1, c3k=False, e=0.5, g=1, shortcut=True):
        """ Initializes the C3k2 module, a faster CSP Bottleneck with 2 convolutions and optional C3k
blocks."""
        super().__init__(c1, c2, n, shortcut, g, e)
        self.m = nn.ModuleList(
C3k(self.c, self.c, 2, shortcut, g) if c3k else Bottleneck_RCM(self.c, self.c,
shortcut, g) for _ in range(n)
)
if __name__ == '__main__':
    TB = RCM(256)
    #创建一个输入张量
    batch_size = 8
    input_tensor=torch.randn(batch_size, 256, 64, 64)
    #运行模型并打印输入和输出的形状
    output_tensor =TB(input_tensor)
    print("Input shape: ", input_tensor.shape)
    print("Output shape:", output_tensor.shape)

```

附录2: 单头空间与层级注意力模块 (SHSA) 实现代码

```
import torch

from .conv import Conv

import torch.nn as nn

from .block import PSABlock, C2PSA

class GroupNorm(torch.nn.GroupNorm):
    """
    Group Normalization with 1 group.
    Input: tensor in shape [B, C, H, W]
    """
    def __init__(self, num_channels, **kwargs):
        super().__init__(1, num_channels, **kwargs)

class Conv2d_BN(torch.nn.Sequential):
    def __init__(self, a, b, ks=1, stride=1, pad=0, dilation=1,
        groups=1, bn_weight_init=1):
        super().__init__()
        self.add_module('c', torch.nn.Conv2d(
            a, b, ks, stride, pad, dilation, groups, bias=False))
        self.add_module('bn', torch.nn.BatchNorm2d(b))
        torch.nn.init.constant_(self.bn.weight, bn_weight_init)
        torch.nn.init.constant_(self.bn.bias, 0)
        @torch.no_grad()
        def fuse(self):
            c, bn = self._modules.values()
            w = bn.weight / (bn.running_var + bn.eps)**0.5
            w = c.weight * w[:, None, None, None]
            b = bn.bias - bn.running_mean * bn.weight / \
                (bn.running_var + bn.eps)**0.5
            m = torch.nn.Conv2d(w.size(1) * self.c.groups, w.size(
                0), w.shape[2:], stride=self.c.stride, padding=self.c.padding,
                dilation=self.c.dilation, groups=self.c.groups,
                device=c.weight.device)
            m.weight.data.copy_(w)
            m.bias.data.copy_(b)
```

```

return m

class SHSA_BLOCK(torch.nn.Module):
    """Single-Head Self-Attention"""
    def __init__(self, dim, qk_dim=16, pdim=32):
        super().__init__()
        self.scale = qk_dim ** -0.5
        self.qk_dim = qk_dim
        self.dim = dim
        self.pdim = pdim
        self.pre_norm = GroupNorm(pdim)
        self.qkv = Conv2d_BN(pdim, qk_dim * 2 + pdim)
        self.proj = torch.nn.Sequential(torch.nn.ReLU(), Conv2d_BN(
            dim, dim, bn_weight_init = 0))
    def forward(self, x):
        B, C, H, W = x.shape
        x1, x2 = torch.split(x, [self.pdim, self.dim - self.pdim], dim = 1)
        x1 = self.pre_norm(x1)
        qkv = self.qkv(x1)
        q, k, v = qkv.split([self.qk_dim, self.qk_dim, self.pdim], dim = 1)
        q, k, v = q.flatten(2), k.flatten(2), v.flatten(2)
        attn = (q.transpose(-2, -1) @ k) * self.scale
        attn = attn.softmax(dim = -1)
        x1 = (v @ attn.transpose(-2, -1)).reshape(B, self.pdim, H, W)
        x = self.proj(torch.cat([x1, x2], dim = 1))
        return x

class PSABlock_SHSA(PSABlock):
    def __init__(self, c, qk_dim=16, pdim=32, shortcut=True) ->None:
        """ Initializes the PSABlock with attention and feed-forward layers for enhanced feature
        extraction. """
        super().__init__(c)
        self.attn = SHSA_BLOCK(c, qk_dim, pdim)
class C2PSA_SHSA(C2PSA):
    def __init__(self, c1, c2, n=1, e=0.5):
        """ Initializes the C2PSA module with specified input/output channels, number of layers ,

```



```

and expansion ratio ."""
super().__init__(c1, c2)
assert c1 == c2
self.c = int(c1 * e)

    self.m = nn.Sequential(*(PSABlock_SHSA(self.c, qk_dim=16, pdim=32) for _ in range(n)))
if __name__ == '__main__':
IDC = SHSA_BLOCK(256, 16, 32)
#创建一个输入张量
batch_size = 8
input_tensor=torch.randn(batch_size, 256, 64, 64)
#运行模型并打印输入和输出的形状
output_tensor =IDC(input_tensor)
print("Input shape: ", input_tensor.shape)
print("Output shape:", output_tensor.shape)
附录3: 对比模块 (Biformer) 实现代码
from typing import Tuple, Optional
import torch
import torch.nn as nn
import torch.nn.functional as F
from einops import rearrange
from torch import Tensor, LongTensor
from .block import PSABlock, C2PSA
class TopkRouting(nn.Module):
    """
    differentiable topk routing with scaling
    Args:
    qk_dim: int , feature dimension of query and key
    topk: int , the 'topk'
    qk_scale: int or None, temperature (multiply) of softmax activation
    with_param: bool, wether incorporate learnable params in routing unit
    diff_routing : bool, wether make routing differentiable
    soft_routing : bool, wether make output value multiplied by routing weights """
    def __init__(self, qk_dim, topk=4, qk_scale=None, param_routing=False, diff_routing=False):
super().__init__()

```

```

self.topk = topk

self.qk_dim = qk_dim

self.scale = qk_scale or qk_dim ** -0.5

self.diff_routing = diff_routing

# TODO: norm layer before/ after linear ?

self.emb = nn.Linear(qk_dim, qk_dim) if param_routing else nn.Identity()

# routing activation

self.routing_act = nn.Softmax(dim=-1)

def forward(self, query: Tensor, key: Tensor) -> Tuple[Tensor]:
    """
    Args:
    q, k: (n, p^2, c) tensor
    Return:
    r_weight, topk_index: (n, p^2, topk) tensor
    """

    if not self.diff_routing :
        query, key = query.detach(), key.detach()

        query_hat, key_hat = self.emb(query), self.emb(key) # per-window pooling ->(n, p^2, c)
        attn_logits = (query_hat * self.scale) @ key_hat.transpose(-2, -1) # (n, p^2, p^2)
        topk_attn_logits, topk_index = torch.topk(attn_logits, k=self.topk, dim=-1) # (n, p^2, k),
        (n, p^2, k)

        r_weight = self.routing_act(topk_attn_logits) # (n, p^2, k)
    return r_weight, topk_index

class KVgather(nn.Module):
    def __init__(self, mul_weight='none'):
        super().__init__()

        assert mul_weight in ['none', 'soft', 'hard']

        self.mul_weight = mul_weight

    def forward(self, r_idx: Tensor, r_weight: Tensor, kv: Tensor):
        """
        r_idx: (n, p^2, topk) tensor
        r_weight: (n, p^2, topk) tensor
        kv: (n, p^2, w^2, c_kq+c_v)

        Return:

```

```

(n, p^2, topk, w^2, c_kq+c_v) tensor
"""

# select kv according to routing index
n, p2, w2, c_kv = kv.size()
topk = r_idx.size(-1)
# print(r_idx.size(), r_weight.size())
# FIXME: gather consumes much memory(topk times redundancy), write cuda kernel?
topk_kv = torch.gather(kv.view(n, 1, p2, w2, c_kv).expand(-1, p2, -1, -1, -1), # (n, p^2, p^2,
w^2, c_kv) without mem cpy
dim=2,
index=r_idx.view(n, p2, topk, 1, 1).expand(-1, -1, -1, w2, c_kv)
# (n, p^2, k, w^2, c_kv)
)
if self.mul_weight == 'soft':
    topk_kv = r_weight.view(n, p2, topk, 1, 1) * topk_kv # (n, p^2, k, w^2, c_kv)
elif self.mul_weight == 'hard':
    raise NotImplementedError('differentiable hard routing TBA')
# else: # 'none'
# topk_kv = topk_kv # do nothing
return topk_kv

class QKVLinear(nn.Module):
    def __init__(self, dim, qk_dim, bias=True):
        super().__init__()
        self.dim = dim
        self.qk_dim = qk_dim
        self.qkv = nn.Linear(dim, qk_dim + qk_dim + dim, bias=bias)
    def forward(self, x):
        q, kv = self.qkv(x).split([self.qk_dim, self.qk_dim + self.dim], dim=-1)
        return q, kv
        # q, k, v = self.qkv(x).split([self.qk_dim, self.qk_dim, self.dim], dim=-1)
# return q, k, v

class BiLevelRoutingAttention(nn.Module):
    """
    n_win: number of windows in one side (so the actual number of windows is n_win*n_win)

```

```

kv_per_win: for kv_downsample_mode='ada_xxxpool' only, number of key/values per window.
Similar to n_win, the actual number is kv_per_win*kv_per_win.
topk: topk for window filtering
param_attention : 'qkvo' - linear for q,k,v and o, 'none': param free attention param_routing: extra
linear for routing
    diff_routing : wether to set routing differentiable
    soft_routing : wether to multiply soft routing weights
"""

def __init__(self, dim, n_win=7, num_heads=8, qk_dim=None, qk_scale=None,
kv_per_win=4, kv_downsample_ratio=4, kv_downsample_kernel=None,
kv_downsample_mode='identity',
    topk=4, param_attention="qkvo", param_routing=False, diff_routing =False,
soft_routing =False,
side_dwconv=3,
auto_pad=True):
    super().__init__()
# local attention setting
self.dim = dim
self.n_win = n_win # Wh, Ww
self.num_heads = num_heads
self.qk_dim = qk_dim or dim
    assert self.qk_dim % num_heads == 0 and self.dim % num_heads == 0, 'qk_dim and dim must
be divisible by num_heads!'
self.scale = qk_scale or self.qk_dim ** -0.5
    #####side_dwconv(i.e. LCE in ShuntedTransformer)#####
    self.lepe = nn.Conv2d(dim, dim, kernel_size=side_dwconv, stride =1, padding=side_dwconv //
2,
groups=dim) if side_dwconv>0 else \
lambda x: torch.zeros_like(x)
    ##### global routing setting #####
self.topk = topk
self.param_routing = param_routing
self.diff_routing = diff_routing
self.soft_routing = soft_routing

```

```

# router
assert not (self . param_routing and not self . diff_routing) # cannot be with_param=True
and diff_routing =False
self . router = TopkRouting(qk_dim=self.qk_dim,
qk_scale=self. scale ,
topk=self.topk,
diff_routing =self . diff_routing ,
param_routing=self . param_routing)
if self . soft_routing : # soft routing , always diffrentiable (if no detach)
mul_weight = ' soft '
elif self . diff_routing : # hard differentiable routing
mul_weight = 'hard'
else: # hard non- differentiable routing
mul_weight = 'none'
self .kv_gather = KVgather(mul_weight=mul_weight)
    # qkv mapping(shared by both global routing and local attention)
self . param_attention = param_attention
if self . param_attention == 'qkvo' :
self .qkv = QKVLinear(self.dim, self .qk_dim)
self .wo = nn.Linear(dim, dim)
elif self .param_attention == 'qkv' :
self .qkv = QKVLinear(self.dim, self .qk_dim)
self .wo = nn. Identity()
else:
    raise ValueError(f' param_attention mode{self. param_attention} is not surpported ! ')
self .kv_downsample_mode = kv_downsample_mode
self .kv_per_win = kv_per_win
self .kv_downsample_ratio = kv_downsample_ratio
self .kv_downsample_kenel = kv_downsample_kernel
if self .kv_downsample_mode == 'ada_avgpool' :
assert self .kv_per_win is not None
    self .kv_down = nn.AdaptiveAvgPool2d(self.kv_per_win)
elif self .kv_downsample_mode == 'ada_maxpool' :
assert self .kv_per_win is not None

```

```

        self.kv_down = nn.AdaptiveMaxPool2d(self.kv_per_win)
    elif self.kv_downsample_mode == 'maxpool':
        assert self.kv_downsample_ratio is not None
        self.kv_down = nn.MaxPool2d(self.kv_downsample_ratio) if self.kv_downsample_ratio>1 else nn.
Identity()
    elif self.kv_downsample_mode == 'avgpool':
        assert self.kv_downsample_ratio is not None
        self.kv_down = nn.AvgPool2d(self.kv_downsample_ratio) if self.kv_downsample_ratio>1 else nn.
Identity()
    elif self.kv_downsample_mode == 'identity' : # no kv downsampling
        self.kv_down = nn.Identity()
    elif self.kv_downsample_mode == 'fracpool':
        # assert self.kv_downsample_ratio is not None
        # assert self.kv_downsample_kernel is not None
        # TODO: fracpool
        # 1. kernel size should be input size dependent
        # 2. there is a random factor, need to avoid independent sampling for k and v raise
NotImplementedError('fracpool policy is not implemented yet! ')
    elif kv_downsample_mode == 'conv':
        # TODO: need to consider the case where k != v so that need two downsample modules raise
NotImplementedError('conv policy is not implemented yet! ')
    else:
        raise ValueError(f'kv_down_sample_mode{self.kv_downsaple_mode}is not surpported!')# softmax for
local attention
    self.attn_act = nn.Softmax(dim=-1)
    self.auto_pad = auto_pad
    def forward(self, x, ret_attn_mask=False):
        """
        x: NHWC tensor
        Return:
        NHWC tensor
        """
        x = rearrange(x, "n c h w ->n h w c")
        # NOTE: use padding for semantic segmentation

```



```

if self .auto_pad:
N, H_in, W_in, C = x. size()
pad_l = pad_t = 0
    pad_r =(self .n_win - W_in % self.n_win)% self.n_win
    pad_b =(self .n_win - H_in % self.n_win)% self.n_win
x = F.pad(x, (0, 0, # dim=-1
pad_l, pad_r, # dim=-2
pad_t, pad_b)) # dim=-3
_, H, W, _ = x. size() # padded size
else:
N, H, W, C = x. size()
    assert H % self.n_win == 0 and W % self.n_win == 0 #
# patchify , (n, p^2, w, w, c), keep 2d window as we need 2d pooling to reduce kv size x = rearrange
(x, "n (j h) (i w)c ->n (j i) h w c", j=self .n_win, i=self .n_win)
#####qkv projection#####
# q: (n, p^2, w, w, c_qk)
# kv: (n, p^2, w, w, c_qk+c_v)
    # NOTE: separte kv if there were memory leak issue caused by gather
q, kv = self .qkv(x)
# pixel-wise qkv
# q_pix: (n, p^2, w^2, c_qk)
# kv_pix: (n, p^2, h_kv*w_kv, c_qk+c_v)
    q_pix = rearrange(q, 'n p2 h w c ->n p2 (h w)c')
    kv_pix = self .kv_down(rearrange(kv, 'n p2 h w c -> (n p2) c h w'))
    kv_pix = rearrange(kv_pix, '(n j i) c h w ->n (j i) (h w)c' , j=self .n_win,
i=self.n_win)
    q_win, k_win = q.mean([2, 3]), kv[ , 0: self .qk_dim].mean(
[2, 3]) # window-wise qk, (n, p^2, c_qk), (n, p^2, c_qk)
#####side_dwconv(lepe)#####
# NOTE: call contiguous to avoid gradient warning when using ddp
    lepe = self .lepe(rearrange(kv[ , self .qk_dim:], 'n (j i) h w c ->n c (j h) (i w)',
j=self.n_win,
i=self.n_win).contiguous())
    lepe = rearrange(lepe , 'n c (j h) (i w) ->n (j h) (i w)c' , j=self .n_win, i=self .n_win)

```

```

##### gather q dependent k/v #####
r_weight, r_idx = self . router(q_win, k_win) # both are (n, p^2, topk) tensors
kv_pix_sel = self .kv_gather(r_idx=r_idx, r_weight=r_weight, kv=kv_pix) # (n, p^2, topk,
h_kv*w_kv, c_qk+c_v)
k_pix_sel , v_pix_sel = kv_pix_sel. split ([self .qk_dim, self .dim], dim=-1)
# kv_pix_sel: (n, p^2, topk, h_kv*w_kv, c_qk)
# v_pix_sel : (n, p^2, topk, h_kv*w_kv, c_v)
##### do attention as normal #####
k_pix_sel = rearrange(k_pix_sel, 'n p2 k w2(m c) -> (n p2)m c(k w2)',
m=self.num_heads) # flatten to BMLC, (n*p^2, m, topk*h_kv*w_kv, c_qk//m)transpose here?
v_pix_sel = rearrange(v_pix_sel, 'n p2 k w2(m c) -> (n p2)m(k w2)c' ,
m=self.num_heads) # flatten to BMLC, (n*p^2, m, topk*h_kv*w_kv, c_v//m)
q_pix = rearrange(q_pix, 'n p2 w2(m c) -> (n p2)m w2 c' ,
m=self.num_heads) # to BMLC tensor(n*p^2, m, w^2, c_qk//m)
# param-free multihead attention
attn_weight =(
q_pix * self .scale)@ k_pix_sel # (n*p^2, m, w^2, c)@(n*p^2,
m, c, topk*h_kv*w_kv)->(n*p^2, m, w^2, topk*h_kv*w_kv)
attn_weight = self . attn_act(attn_weight)
out = attn_weight @ v_pix_sel # (n*p^2, m, w^2, topk*h_kv*w_kv)@(n*p^2, m,
topk*h_kv*w_kv, c)->(n*p^2, m, w^2, c)
out = rearrange(out, '(n j i)m(h w)c ->n (j h) (i w) (m c)' , j=self .n_win,
i=self.n_win,
h=H // self .n_win, w=W // self .n_win)
out = out + lepe
# output linear
out = self .wo(out)
# NOTE: use padding for semantic segmentation
# crop padded region
if self .auto_pad and(pad_r>0 or pad_b>0):
out = out[:, :H_in, :W_in, :]. contiguous()
if ret_attn_mask:
return out, r_weight, r_idx, attn_weight
else:

```

```

return rearrange(out, "n h w c ->n c h w")

class Attention(nn.Module):
    """
    vanilla attention
    """

    def __init__(self, dim, num_heads=8, qkv_bias=False, qk_scale=None, attn_drop=0.,
proj_drop=0.):
    super().__init__()
    self.num_heads = num_heads
    head_dim = dim // num_heads

    # NOTE scale factor was wrong in my original version , can set manually to be compat with
prev weights
    self.scale = qk_scale or head_dim ** -0.5
    self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
    self.attn_drop = nn.Dropout(attn_drop)
    self.proj = nn.Linear(dim, dim)
    self.proj_drop = nn.Dropout(proj_drop)

    def forward(self, x):
    """
    args:
    x: NCHW tensor

    return :
    NCHW tensor
    """

    _, _, H, W = x.size()
    x = rearrange(x, 'n c h w ->n (h w) c')
    B, N, C = x.shape

    qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, C // self.num_heads).permute(2, 0, 3, 1, 4)
    q, k, v = qkv[0], qkv[1], qkv[2] # make torchscript happy (cannot use tensor as tuple)
    attn = (q @ k.transpose(-2, -1)) * self.scale
    attn = attn.softmax(dim=-1)
    attn = self.attn_drop(attn)
    x = (attn @ v).transpose(1, 2).reshape(B, N, C)
    x = self.proj(x)

```

```

x = self.proj_drop(x)

x = rearrange(x, 'n (h w) c ->n c h w', h=H, w=W)

return x

class AttentionLePE(nn.Module):
    """
    vanilla attention
    """

    def __init__(self, dim, num_heads=8, qkv_bias=False, qk_scale=None, attn_drop=0.,
proj_drop=0., side_dwconv=5):
    super().__init__()
    self.num_heads = num_heads
    head_dim = dim // num_heads

    # NOTE scale factor was wrong in my original version , can set manually to be compat with prev
weights
    self.scale = qk_scale or head_dim ** -0.5
    self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
    self.attn_drop = nn.Dropout(attn_drop)
    self.proj = nn.Linear(dim, dim)
    self.proj_drop = nn.Dropout(proj_drop)

    self.lepe = nn.Conv2d(dim, dim, kernel_size=side_dwconv, stride =1, padding=side_dwconv //2,
groups=dim) if side_dwconv>0 else \
lambda x: torch.zeros_like(x)

    def forward(self, x):
    """
    args:
    x: NCHW tensor

    return :
    NCHW tensor
    """

    _, _, H, W = x.size()
    x = rearrange(x, 'n c h w ->n (h w) c')

    B, N, C = x.shape

    qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, C // self.num_heads).permute(2, 0, 3, 1, 4)
    q, k, v = qkv[0], qkv[1], qkv[2] # make torchscript happy (cannot use tensor as tuple)

```

```

        lepe = self.lepe(rearrange(x, 'n (h w)c ->n c h w', h=H, w=W))
lepe = rearrange(lepe, 'n c h w ->n (h w)c')
attn=(q @ k.transpose(-2, -1)) * self.scale
attn = attn.softmax(dim=-1)
attn = self.attn_drop(attn)
x=(attn @ v).transpose(1, 2).reshape(B, N, C)
x = x + lepe
x = self.proj(x)
x = self.proj_drop(x)
        x = rearrange(x, 'n (h w)c ->n c h w', h=H, w=W)
return x

    def _grid2seq(x: Tensor, region_size : Tuple[int ], num_heads: int):
        """
        Args:
        x: BCHW tensor
        region_size : int
        num_heads: number of attention heads
        Return:
            out: rearranged x, has a shape of (bs, nhead, nregion, reg_size , head_dim)
        region_h, region_w: number of regions per col/row
        """
        B, C, H, W = x.size()
        region_h, region_w = H // region_size[0], W // region_size[1]
        x = x.view(B, num_heads, C // num_heads, region_h, region_size[0], region_w, region_size[1])
        x = torch.einsum('bmdhpwq->bmhwpqd', x).flatten(2, 3).flatten(-3, -2) # (bs, nhead, nregion,
        reg_size , head_dim)
        return x, region_h, region_w

    def _seq2grid(x: Tensor, region_h: int , region_w: int , region_size : Tuple[int]):
        """
        Args:
        x: (bs, nhead, nregion, reg_size ^2, head_dim)
        Return:
        x: (bs, C, H, W)
        """

```

```

    bs, nhead, nregion, reg_size_square , head_dim = x.size()

    x = x.view(bs, nhead, region_h, region_w, region_size[0], region_size[1], head_dim)

    x = torch.einsum('bmhwpqd->bmdhpwq', x).reshape(bs, nhead * head_dim,
region_h * region_size[0], region_w *
region_size[1])

return x

def regional_routing_attention_torch (
    query: Tensor, key: Tensor, value: Tensor, scale: float ,
    region_graph: LongTensor, region_size : Tuple[int],
kv_region_size: Optional[Tuple[int]] = None,
auto_pad=True) ->Tensor:
    """
    Args:
    query, key, value: (B, C, H, W)tensor
        scale : the scale /temperature for dot product attention
        region_graph: (B, nhead, h_q*w_q, topk) tensor , topk<= h_k*w_k
        region_size : region/window size for queries , (rh, rw)
        key_region_size : optional , if None, key_region_size=region_size
        auto_pad: required to be true if the input sizes are not divisible by the region_size Return:
    output: (B, C, H, W) tensor
    attn : (bs, nhead, q_nregion, reg_size , topk*kv_region_size) attention matrix """
    kv_region_size = kv_region_size or region_size
    bs, nhead, q_nregion, topk = region_graph.size()
    # Auto pad to deal with any input size
    q_pad_b, q_pad_r, kv_pad_b, kv_pad_r = 0, 0, 0, 0
    if auto_pad:
        _, _, Hq, Wq = query.size()
        q_pad_b =(region_size[0] - Hq % region_size[0])% region_size[0]
        q_pad_r =(region_size[1] - Wq % region_size[1])% region_size[1]
    if (q_pad_b>0 or q_pad_r>0):
        query = F.pad(query, (0, q_pad_r, 0, q_pad_b)) # zero padding
    _, _, Hk, Wk = key.size()
    kv_pad_b =(kv_region_size[0] - Hk % kv_region_size[0])% kv_region_size[0]
    kv_pad_r =(kv_region_size[1] - Wk % kv_region_size[1])% kv_region_size[1]

```

```

if (kv_pad_r>0 or kv_pad_b>0):
    key = F.pad(key, (0, kv_pad_r, 0, kv_pad_b)) # zero padding
    value = F.pad(value, (0, kv_pad_r, 0, kv_pad_b)) # zero padding
    # to sequence format, i .e. (bs, nhead, nregion, reg_size , head_dim)
    query, q_region_h, q_region_w = _grid2seq(query, region_size=region_size, num_heads=nhead)key,
    _, _ = _grid2seq(key, region_size=kv_region_size, num_heads=nhead)
    value, _, _ = _grid2seq(value, region_size=kv_region_size, num_heads=nhead)
# gather key and values .
    # TODO: is seperate gathering slower than fused one(our old version) ?
# torch .gather does not support broadcasting , hence we do it manually
    bs, nhead, kv_nregion, kv_region_size, head_dim = key.size()
broadcasted_region_graph = region_graph.view(bs, nhead, q_nregion, topk, 1, 1). \
    expand(-1, -1, -1, -1, kv_region_size, head_dim)
    key_g = torch .gather(key.view(bs, nhead, 1, kv_nregion, kv_region_size, head_dim). \
    expand(-1, -1, query. size(2), -1, -1, -1), dim=3,
index=broadcasted_region_graph) # (bs, nhead, q_nregion, topk,
kv_region_size, head_dim)
    value_g = torch .gather(value.view(bs, nhead, 1, kv_nregion, kv_region_size, head_dim). \expand
(-1, -1, query. size(2), -1, -1, -1), dim=3,
index=broadcasted_region_graph) # (bs, nhead, q_nregion, topk,
kv_region_size, head_dim)
# token-to-token attention
    # (bs, nhead, q_nregion, reg_size, head_dim)@(bs, nhead, q_nregion, head_dim,
topk*kv_region_size)
    # ->(bs, nhead, q_nregion, reg_size , topk*kv_region_size)
# TODO: mask padding region
    attn =(query * scale)@ key_g.flatten(-3, -2). transpose(-1, -2)
attn = torch.softmax(attn , dim=-1)
    # (bs, nhead, q_nregion, reg_size, topk*kv_region_size)@(bs, nhead, q_nregion,
topk*kv_region_size , head_dim)
    # ->(bs, nhead, q_nregion, reg_size , head_dim)
output = attn @ value_g. flatten (-3, -2)
# to BCHW format
    output = _seq2grid(output , region_h=q_region_h, region_w=q_region_w, region_size=region_size)

```



```

# remove paddings if needed
if auto_pad and (q_pad_b>0 or q_pad_r>0):
    output = output[:, :, :Hq, :Wq]
return output, attn

class PSABlock_biformer(PSABlock):
    def __init__(self, c, qk_dim=16, pdim=32, shortcut=True) ->None:
        """ Initializes the PSABlock with attention and feed-forward layers for enhanced feature
        extraction. """
        super().__init__(c)
        self.attn = BiLevelRoutingAttention(c, qk_dim, pdim)
        class C2PSA_biformer(C2PSA):
            def __init__(self, c1, c2, n=1, e=0.5):
                """ Initializes the C2PSA module with specified input/output channels, number of layers, and
                expansion ratio. """
                super().__init__(c1, c2)
                assert c1 == c2
                self.c = int(c1 * e)
                self.m = nn.Sequential(*(PSABlock_biformer(self.c, qk_dim=16, pdim=32) for _ in
                range(n)))

if __name__ == '__main__':

```

```

IDC = BiLevelRoutingAttention(256)
#创建一个输入张量
batch_size = 8
input_tensor=torch.randn(batch_size, 256, 64, 64)
#运行模型并打印输入和输出的形状
output_tensor =IDC(input_tensor)
print("Input shape: ", input_tensor.shape)
print("Output shape:", output_tensor.shape)

```

附录4：武器智能检测平台主程序（基于PyQt6）

```

import sys
import cv2
import time
from ultralytics import YOLO

```

```

import torch

from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QPushButton, QLabel, QFileDialog, QVBoxLayout,
    QWidget, QTextEdit, QStackedWidget, QHBoxLayout, QGridLayout
)

from PyQt6.QtGui import QPixmap, QImage, QFont
from PyQt6.QtCore import Qt, QTimer

WEIGHT_PATH = r"./best.pt"

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

def get_color(idx):
    idx = idx * 3
    return ((37 * idx) % 255, (17 * idx) % 255, (29 * idx) % 255)

class Detector:
    def __init__(self, weight_path, conf_threshold=0.5, iou_threshold=0.5):
        self.device = device
        self.model = YOLO(weight_path)
        self.conf_threshold = conf_threshold
        self.iou_threshold = iou_threshold
        self.names = self.model.names

    def detect_image(self, img_bgr):
        results = self.model(img_bgr, verbose=False, conf=self.conf_threshold,
                               iou=self.iou_threshold, device=self.device)
        bboxes_cls = results[0].boxes.cls
        bboxes_conf = results[0].boxes.conf

        bboxes_xyxy = results[0].boxes.xyxy.cpu().numpy().astype('uint32')
        detection_details = []
        for idx in range(len(bboxes_cls)):
            box_conf = f"{bboxes_conf[idx]:.2f}"
            box_cls = int(bboxes_cls[idx])
            bbox_xyxy = bboxes_xyxy[idx]
            bbox_label = self.names[box_cls]
            xmin, ymin, xmax, ymax = bbox_xyxy

            img_bgr = cv2.rectangle(img_bgr, (xmin, ymin), (xmax, ymax), get_color(box_cls + 3), 2)
            label_y = int(max(int(ymin) - 10, 0))

```

```

cv2.putText(img_bgr, f' {bbox_label} ({box_conf})', (int(xmin), label_y),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, get_color(box_cls + 3), 2)
detection_details.append(f"类别: {bbox_label}, 置信度: {box_conf}, 框: [{xmin},
{ymin}, {xmax}, {ymax}]")
return img_bgr, detection_details

class StyledButton(QPushButton):
    def __init__(self, text):
        super().__init__(text)
        self.setFont(QFont("Microsoft YaHei", 16))
        self.setMinimumSize(200, 200)
        self.setStyleSheet("""
QPushButton{
background-color: #2D8CFF
color : white
border-radius: 10px
font-size: 20px
}
QPushButton: hover{
background-color: #1E6ED8
}
""")

class MainMenu(QWidget):
    def __init__(self, stacked_widget):
        super().__init__()
        self.stacked_widget = stacked_widget
        layout = QGridLayout()
        layout.setSpacing(40)
        layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
        btn_img = StyledButton("图片识别")
        btn_img.clicked.connect(lambda: self.stacked_widget.setCurrentIndex(1))
        btn_vid = StyledButton("视频识别")
        btn_vid.clicked.connect(lambda: self.stacked_widget.setCurrentIndex(2))
        btn_cam = StyledButton("摄像头识别")
        btn_cam.clicked.connect(lambda: self.stacked_widget.setCurrentIndex(3))
        btn_exit = StyledButton("退出程序")

```

```

btn_exit . clicked . connect ( sys . exit )

layout . addWidget ( btn_img , 0 , 0 )
layout . addWidget ( btn_vid , 0 , 1 )
layout . addWidget ( btn_cam , 1 , 0 )
layout . addWidget ( btn_exit , 1 , 1 )

self . setLayout ( layout )

class ImageRecognitionPage ( QWidget ) :
    def __init__ ( self , stacked_widget ) :
        super () . __init__ ()
        self . stacked_widget = stacked_widget
        self . detector = Detector ( weight_path = WEIGHT_PATH , conf_threshold = 0.2 , iou_threshold = 0.5 )
        self . result_image = None
        self . original_image = None
        layout = QVBoxLayout ()
        content_layout = QHBoxLayout ()
        right_layout = QVBoxLayout ()
        button_layout = QHBoxLayout ()
        self . original_image_label = QLabel ( "原图显示区域" )
            self . original_image_label . setAlignment ( Qt . AlignmentFlag . AlignCenter )
            self . original_image_label . setMinimumSize ( 500 , 600 )
            self . original_image_label . setMaximumSize ( 500 , 600 )
        self . original_image_label . setStyleSheet ( "border: 2px solid blue" )
        self . detected_image_label = QLabel ( "检测图显示区域" )
            self . detected_image_label . setAlignment ( Qt . AlignmentFlag . AlignCenter )
            self . detected_image_label . setMinimumSize ( 500 , 400 )
        self . detected_image_label . setStyleSheet ( "border: 2px solid red" )
        self . result_text = QTextEdit ()
        self . result_text . setReadOnly ( True )
        self . result_text . setMinimumSize ( 500 , 200 )
        self . result_text . setStyleSheet ( "border: 2px solid red" )
            content_layout . addWidget ( self . original_image_label , alignment = Qt . AlignmentFlag . AlignTop )
            right_layout . addWidget ( self . detected_image_label )
        right_layout . addWidget ( self . result_text )
        content_layout . addLayout ( right_layout )

```

```

btn_upload = StyledButton("上传图片")
btn_upload.clicked.connect(self.open_image)
btn_export = StyledButton("导出结果")
    btn_export.clicked.connect(self.export_result)
btn_back = StyledButton("返回")
    btn_back.clicked.connect(lambda: self.stacked_widget.setCurrentIndex(0))
button_layout.addWidget(btn_upload)
button_layout.addWidget(btn_export)
button_layout.addWidget(btn_back)
layout.addLayout(content_layout)
layout.addLayout(button_layout)
self.setLayout(layout)
def open_image(self):
    file_name, _ = QFileDialog.getOpenFileName(self, "选择图片", "", "Images(*.png *.jpg *.jpeg)")
    if file_name:
        img_bgr = cv2.imread(file_name)
        self.original_image = img_bgr.copy()
        img_bgr, results = self.detector.detect_image(img_bgr)
        self.result_image = img_bgr
        self.display_images(self.original_image, img_bgr)
        self.display_results(results)
    def display_images(self, original_img, detected_img):
# Display original image
        img_rgb = cv2.cvtColor(original_img, cv2.COLOR_BGR2RGB)
        h, w, ch = img_rgb.shape
        qt_image = QImage(img_rgb.data, w, h, ch * w, QImage.Format.Format_RGB888)
        pixmap = QPixmap.fromImage(qt_image)
        self.original_image_label.setPixmap(pixmap.scaled(self.original_image_label.size(),
Qt.AspectRatioMode.KeepAspectRatio))
# Display detected image
        img_rgb = cv2.cvtColor(detected_img, cv2.COLOR_BGR2RGB)
        h, w, ch = img_rgb.shape
        qt_image = QImage(img_rgb.data, w, h, ch * w, QImage.Format.Format_RGB888)

```

```

pixmap = QPixmap.fromImage(qt_image)

self . detected_image_label .setPixmap(pixmap.scaled(self .detected_image_label .size(),Qt.
AspectRatioMode.KeepAspectRatio))

def display_results (self , results):
self . result_text . clear()

for r in results :
self . result_text .append(r)

def export_result (self):
if self .result_image is not None:
file_name, _ = QFileDialog.getSaveFileName(self, "保存结果", "", "Images (*.png *.jpg
*.jpeg)")
if file_name:
cv2.imwrite(file_name, self . result_image)

class VideoRecognitionPage(QWidget):
def __init__(self , stacked_widget):
super(). __init__()
self .stacked_widget = stacked_widget
self . detector = Detector(weight_path=WEIGHT_PATH)
self . result_video =[]
self . original_video =[]
self .video_paused = False
layout = QVBoxLayout()
content_layout = QHBoxLayout()
right_layout = QVBoxLayout()
button_layout = QHBoxLayout()
self . original_image_label = QLabel("原图显示区域")
self . original_image_label .setAlignment(Qt.AlignmentFlag.AlignCenter)
self . original_image_label .setMinimumSize(500, 600)
self . original_image_label .setMaximumSize(500, 600)
self . original_image_label . setStyleSheet ("border: 2px solid blue")
self . detected_image_label = QLabel("检测图显示区域")
self . detected_image_label .setAlignment(Qt.AlignmentFlag.AlignCenter)
self . detected_image_label .setMinimumSize(500, 400)
self . detected_image_label . setStyleSheet ("border: 2px solid red")

```

```

self . result_text = QTextEdit()
self . result_text . setReadOnly(True)
self . result_text . setMinimumSize(500, 200)
self . result_text . setStyleSheet("border: 2px solid red")
    content_layout . addWidget(self. original_image_label , alignment=Qt.AlignmentFlag.AlignTop)
    right_layout . addWidget(self. detected_image_label)
right_layout . addWidget(self. result_text )
content_layout . addLayout(right_layout)
btn_upload = StyledButton("上传视频")
btn_upload. clicked . connect(self . open_video)
btn_pause = StyledButton("暂停检测")
btn_pause. clicked . connect(self . pause_video)
btn_resume = StyledButton("继续检测")
btn_resume.clicked . connect(self . resume_video)
btn_export = StyledButton("导出视频")
    btn_export. clicked . connect(self . export_result )
btn_back = StyledButton("返回")
    btn_back.clicked . connect(lambda: self . stacked_widget. setCurrentIndex(0))
button_layout . addWidget(btn_upload)
button_layout . addWidget(btn_pause)
button_layout . addWidget(btn_resume)
button_layout . addWidget(btn_export)
button_layout . addWidget(btn_back)
layout.addLayout(content_layout)
layout.addLayout(button_layout)
self . setLayout(layout)
def open_video(self):
    file_name, _ = QFileDialog.getOpenFileName(self, "选择视频", "", "Videos (*.mp4 *.avi
*.mov)")
    if file_name:
        cap = cv2.VideoCapture(file_name)
        self . result_video . clear ()
        self . original_video . clear ()
        self . video_paused = False

```



```

while cap.isOpened():
    if self.video_paused:
        QApplication.processEvents()
        continue
    ret, frame = cap.read()
    if not ret:
        break
    self.original_video.append(frame.copy())
    frame, results = self.detector.detect_image(frame)
    self.result_video.append(frame)
    self.display_images(self.original_video[-1], frame)
    self.display_results(results)
    QApplication.processEvents()
    time.sleep(0.03)
    cap.release()
    def display_images(self, original_img, detected_img):
# Display original image
        img_rgb = cv2.cvtColor(original_img, cv2.COLOR_BGR2RGB)
        h, w, ch = img_rgb.shape
        qt_image = QImage(img_rgb.data, w, h, ch * w, QImage.Format.Format_RGB888)
        pixmap = QPixmap.fromImage(qt_image)
        self.original_image_label.setPixmap(pixmap.scaled(self.original_image_label.size(),
Qt.AspectRatioMode.KeepAspectRatio))
# Display detected image
        img_rgb = cv2.cvtColor(detected_img, cv2.COLOR_BGR2RGB)
        h, w, ch = img_rgb.shape
        qt_image = QImage(img_rgb.data, w, h, ch * w, QImage.Format.Format_RGB888)
        pixmap = QPixmap.fromImage(qt_image)
        self.detected_image_label.setPixmap(pixmap.scaled(self.detected_image_label.size(), Qt.
AspectRatioMode.KeepAspectRatio))
    def display_results(self, results):
        self.result_text.clear()
        for r in results:
            self.result_text.append(r)

```

```

def pause_video(self):
    self.video_paused = True

def resume_video(self):
    self.video_paused = False

def export_result(self):
    if self.result_video:
        file_name, _ = QFileDialog.getSaveFileName(self, "保存视频", "", "Videos (*.mp4)")
        if file_name:
            height, width, _ = self.result_video[0].shape
            out = cv2.VideoWriter(file_name, cv2.VideoWriter_fourcc(*'mp4v'), 20.0, (width,height))
            for frame in self.result_video:
                out.write(frame)
            out.release()

class CameraRecognitionPage(QWidget):
    def __init__(self, stacked_widget):
        super().__init__()
        self.stacked_widget = stacked_widget
        self.detector = Detector(weight_path=WEIGHT_PATH)
        self.cap = None
        self.timer = QTimer()
        self.timer.timeout.connect(self.next_frame)
        layout = QVBoxLayout()
        content_layout = QHBoxLayout()
        right_layout = QVBoxLayout()
        button_layout = QHBoxLayout()
        self.original_image_label = QLabel("原图显示区域")
        self.original_image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.original_image_label.setMinimumSize(500, 600)
        self.original_image_label.setMaximumSize(500, 600)
        self.original_image_label.setStyleSheet("border: 2px solid blue")
        self.detected_image_label = QLabel("检测图显示区域")
        self.detected_image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.detected_image_label.setMinimumSize(500, 400)
        self.detected_image_label.setStyleSheet("border: 2px solid red")

```

```

self . result_text = QTextEdit()
self . result_text . setReadOnly(True)
self . result_text . setMinimumSize(500, 200)
self . result_text . setStyleSheet("border: 2px solid red")
    content_layout . addWidget(self. original_image_label , alignment=Qt.AlignmentFlag.AlignTop)
right_layout . addWidget(self. detected_image_label)
right_layout . addWidget(self. result_text )
content_layout . addLayout(right_layout)
btn_start = StyledButton("启动摄像头")
btn_start . clicked . connect(self . start_camera)
btn_stop = StyledButton("停止摄像头")
btn_stop. clicked . connect(self . stop_camera)
btn_capture = StyledButton("保存截图")
    btn_capture . clicked . connect(self . capture_frame)
btn_back = StyledButton("返回")
btn_back.clicked . connect(self . go_back)
button_layout . addWidget(btn_start)
button_layout . addWidget(btn_stop)
button_layout . addWidget(btn_capture)
button_layout . addWidget(btn_back)
layout.addLayout(content_layout)
layout.addLayout(button_layout)
self . setLayout(layout)
def start_camera(self):
if self . cap is None or not self . cap.isOpened():
self . cap = cv2.VideoCapture(0)
if not self . cap.isOpened():
self . result_text . setText("无法打开摄像头")
self . cap = None
return
self . timer. start (30)
def stop_camera(self):
if self . cap is not None:
self . timer.stop()

```

```

self .cap. release()

self .cap = None

self . original_image_label . clear ()

self . original_image_label . setText("原图显示区域")

self . detected_image_label. clear ()

self . detected_image_label. setText("检测图显示区域")

self . result_text . clear()

def capture_frame(self):

if hasattr (self , ' current_detected_frame ') and self . current_detected_frame is not None:

    file_name, _ = QFileDialog.getSaveFileName(self, "保存截图", "", "Images (*.png *.jpg

*.jpeg)")

if file_name:

    cv2.imwrite(file_name, self . current_detected_frame)

def go_back(self):

self .stop_camera()

self .stacked_widget. setCurrentIndex(0)

def next_frame(self):

    if self .cap and self .cap.isOpened():ret , frame = self .cap.read()

if ret :

self . current_original_frame = frame.copy()

    self . current_detected_frame , results = self . detector .detect_image(frame)

    self .display_images(self . current_original_frame , self . current_detected_frame)

self . display_results ( results)

    def display_images(self , original_img, detected_img):

# Display original image

    img_rgb = cv2.cvtColor(original_img, cv2.COLOR_BGR2RGB)

h, w, ch = img_rgb.shape

    qt_image = QImage(img_rgb.data, w, h, ch * w, QImage.Format.Format_RGB888)

pixmap = QPixmap.fromImage(qt_image)

    self . original_image_label .setPixmap(pixmap.scaled(self . original_image_label .size(),

Qt.AspectRatioMode.KeepAspectRatio))

# Display detected image

    img_rgb = cv2.cvtColor(detected_img, cv2.COLOR_BGR2RGB)

h, w, ch = img_rgb.shape

```

```

qt_image = QImage(img_rgb.data, w, h, ch * w, QImage.Format.Format_RGB888)
pixmap = QPixmap.fromImage(qt_image)

self . detected_image_label .setPixmap(pixmap.scaled(self .detected_image_label .size(),Qt.
AspectRatioMode.KeepAspectRatio))

def display_results (self , results):
self . result_text . clear()

for r in results :
self . result_text .append(r)

class MainWindow(QMainWindow):
def __init__(self):
super(). __init__()

self .setWindowTitle("YOLO武器检测系统")
self .setGeometry(100, 100, 1080, 800)
self .stacked_widget = QStackedWidget(self)

self .stacked_widget.addWidget(MainMenu(self.stacked_widget))
self .stacked_widget.addWidget(ImageRecognitionPage(self.stacked_widget))
self .stacked_widget.addWidget(VideoRecognitionPage(self.stacked_widget))
self .stacked_widget.addWidget(CameraRecognitionPage(self.stacked_widget))

self . setCentralWidget(self .stacked_widget)

if __name__ == "__main__":
app = QApplication(sys.argv)

window = MainWindow()

window.show()

sys. exit(app.exec())

```

致 谢

行文至此，落笔为终。随着论文的完成，我的本科生涯也将正式画上句号。四年的光阴如白驹过隙，恍然之间，那个初入校园、懵懂青涩的自己已蜕变为能够独当一面的青年。目光所及，皆是回忆；心之所向，满是感恩。在赤花山下的四年，是我人生中最宝贵、最青春的一段时光。纵有万般不舍，亦终将远行，唯有满怀感激，铭记于心。

首先，衷心感谢我的父母与亲人。你们是我顺境中的坚定支持，是我逆境时的坚实后盾。二十余载春秋寒暑，你们用心守护、默默付出，以最大的包容理解我的成长，以无私的信任成就我的选择。正如那句诗所言：“无论黄昏将树的影子拉得多长，它总是和根连在一起。”树犹如此，我亦如此。无论未来走到哪里，心中都清楚：身后那盏不灭的灯光，永远为我点亮归途，也温暖前行。

二是感谢我大学期间的每一位老师。你们用知识浇灌我思维的田地，用耐心引导我走向学术的彼岸。课堂上的

点拨、论文中的指正，生活中的关怀，都让我深刻体会到“师者，所以传道、授业、解惑也”的真正含义。你们不仅是知识的传播者，更是人生旅途中的引路人。特别感谢我的毕业论文指导老师，您严谨治学的态度、悉心指导的精神，为我打开了科研思维的大门。在论文撰写过程中，您耐心细致地为我指出问题、提供方向，既不放松要求，也始终给予我鼓励。这份学术上的支持与信任，将长久铭刻于我心。

三是感谢我的同窗与朋友们。我们一同经历课堂的思辨与考场的洗礼，共享图书馆的寂静与操场的欢笑。你们的陪伴，是我青春岁月中最闪亮的篇章。在迷茫时相互鼓励，在困难时携手前行，那些关于梦想与热爱的讨论，将是我此生难忘的回忆。

最后，我要感谢那个在逆境中始终没有放弃、始终努力坚持的自己。在科研中一次次调试失败的深夜，在迷茫无助时也咬牙坚持的日子，都是我成长的见证。感谢自己的勇气与坚韧，让我有能力走到今天，也有底气迈向未来。

在赤花山下度过的四年，不只是学习知识的过程，更是认识自我、锤炼心志，追寻理想的过程。这段旅程虽已抵达终点，但它给予我的力量，将陪伴我继续走向下一段未知的远方。愿此去前程似锦，归来仍是少年。

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告结合AI大模型自动生成，结果仅供参考。
- 报告支持中、英文内容检测。



微信公众号

唯一官网: <https://vpcs.fanyu.com> | 客服邮箱: vpcs@fanyu.com | 客服热线: 400-607-5550 | 客服QQ: 4006075550